ML PRIMER FOR SOFTWARE SPECIALISTS

02.07.2026

VeeMG x adnexo

Martin Stypinski

- Partner @ VeeMG GmbH
  - Software & AI Consulting
- Dean of Studies :
  - CAS Machine Learning 4 Software Engineers
- Hobbies: Stupid Ideas & Cycling



Christian Fässler

- Co-Founder @ adnexo GmbH
  - Fleet Telematics
  - Sensor / Timeseries Data
  - Pythonista
- Teacher :
  - CAS Machine Learning 4 Software Engineers
- Hobbies: Sports & Music



Let’s connect

Martin Stypinski

https://veemg.com

Christian Fässler

www.adnexo.ch

Time Table - Morning

- 09:00 – 12:00: Introduction into ML
  - What is AI, ML, DL
  - Understanding Features, Bias Variance Trade-off, IID assumption

Time Table - Afternoon

- 13:30 – 15:00: Classical Methods with SciKit Learn
  - Trees & Classification
  - Lab Session

- 15:30 – 17:00: Deep Learning
  - Introduction into Deep Learning
  - Deep Learning Lab

- Goal for today: **Train a model, and tinkering!**

Time Table - Tomorrow

- 09:00 – 11:00: Deep Learning Continued
  - Transfer Learning
  - Lab Session
- 11:00 – 12:00 MLOps & Project Intro
- 13:00 – 16:00: Lab
- 16:00 – 17:00: From Idea to Business

- Goal for tomorrow: **Take home a working blueprint project!**

Goal for Today

1. Understand Supervised & Un-Supervised Learning
2. What is needed for a successful ML Project
3. Get our hands dirty! Doing!
4. Fun!

Outline

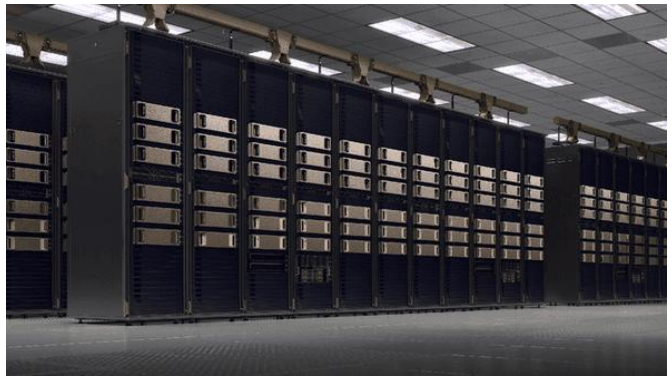
- «Big Data» as a Phenomenon

- AI, ML, DL – 3 terms, but what is the ”deeper” meaning.
- Some examples of modern Machine Learning Projects
  - Industry Drivers

Data Mining

- Automatic extraction of valuable knowledge from large datasets.
- Support human decision-making.
  - Hmm, GPT?







Machine Learning

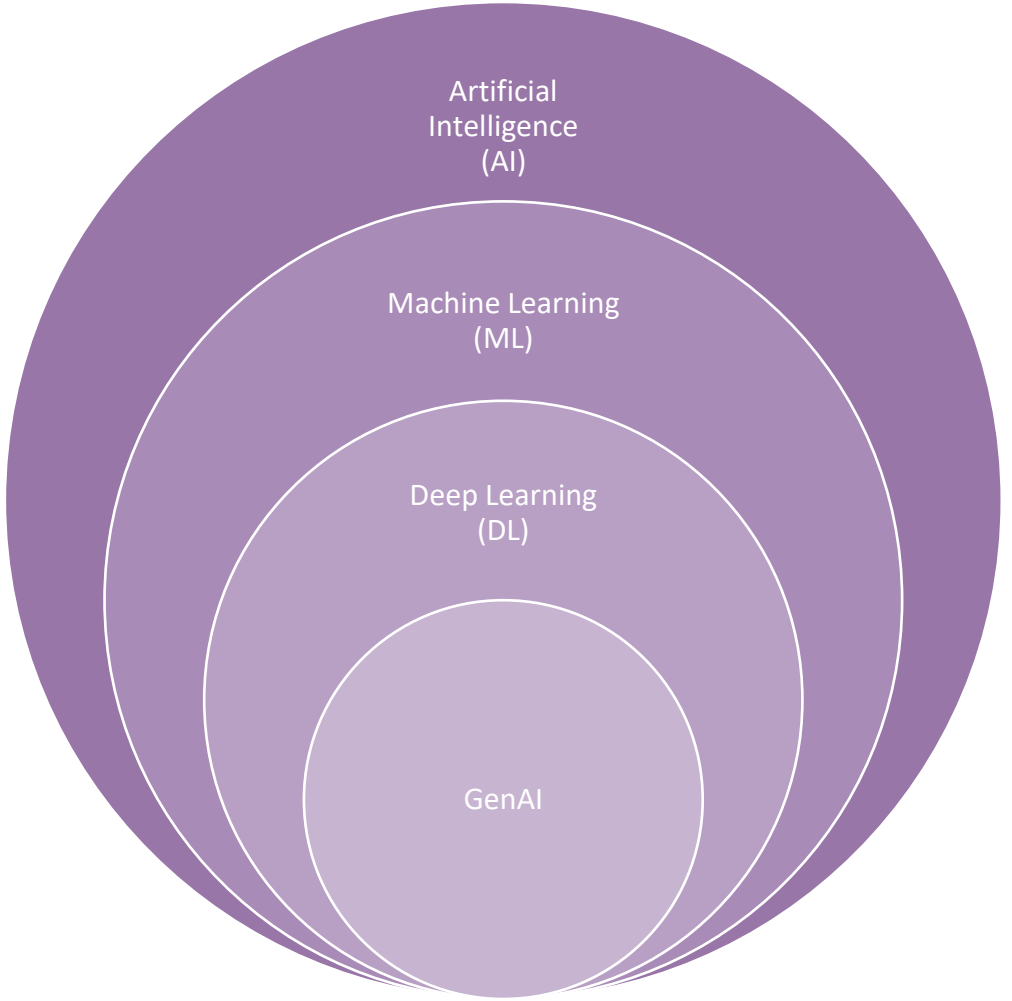
- Use computers to recognize **patterns** in data and make **predictions** or decisions **automatically**.

- Useful in case of:
  - Automation of human processes.
  - «Beyond Human Scope» - Ever looked through 1 TB of data? 😳



AI, ML & DL?

- Artificial Intelligence (AI)
  - Intelligent Agents
  - Deterministic!
- Machine Learning (ML)
  - Collection of Mathematical Methods to find **Patterns** in Data.
- Deep Learning (DL)
  - Machine Learning with Neuronal Networks
  - Still Deterministic; but *Fuzzy*
- GenAI
- Agentic



Samples for AI, ML, DL

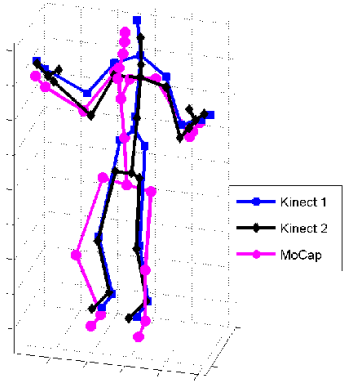
- Use Cases for AI:
  - Agent Systems (Google Maps, Board Games)
  - CSP Problems (Stundenplan)
- Use Cases for ML:
  - Regression Methods (Forecasting)
  - Random Forest (Loan, Morgage Prediction)
- Use Cases for DL:
  - Semantic Image Recognition
  - Language Understanding, Translation
  - Reinforcement Learning (Computer Games)
  - ChatGPT (GPT-4 & Falcon-44B)

Muffin or Chihuahua?

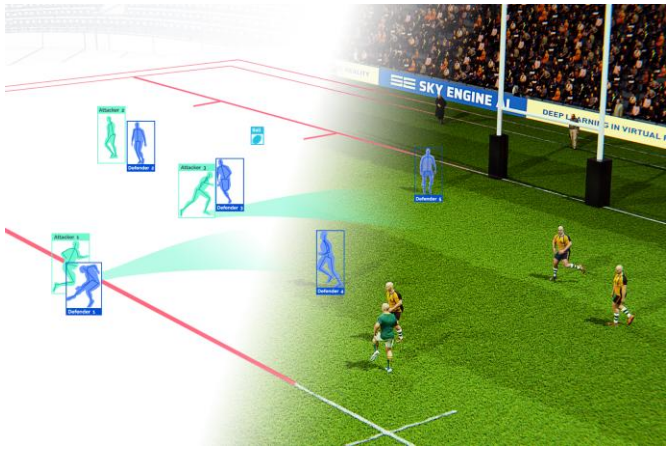


More Examples for ML Use-Cases

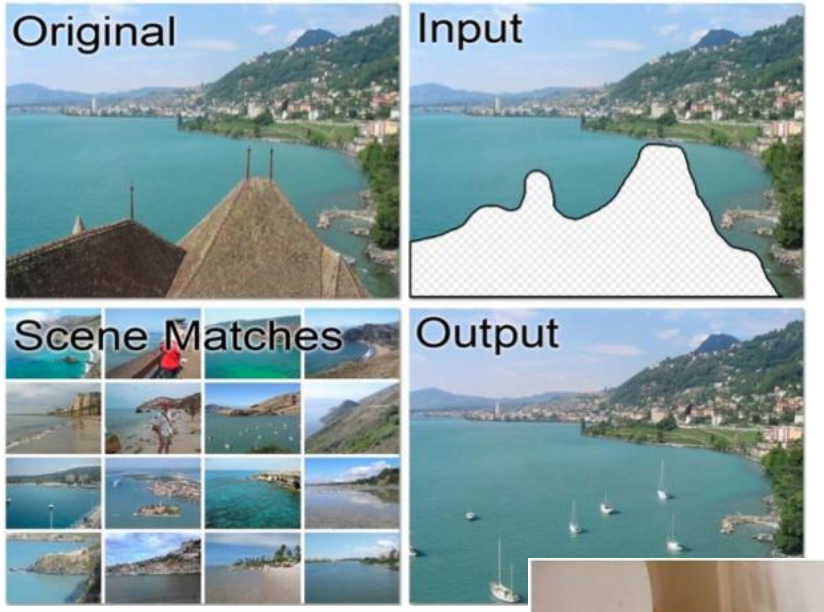
- Motion Capture
- Quality Control (Video)
- Movement Analysis (Hockey, Soccer)







More Examples




a cat is sitting on a toilet seat
logprob: -7.79


a display case filled with lots of different types of donuts
logprob: -7.78



Driver of the Deep Learning Movement

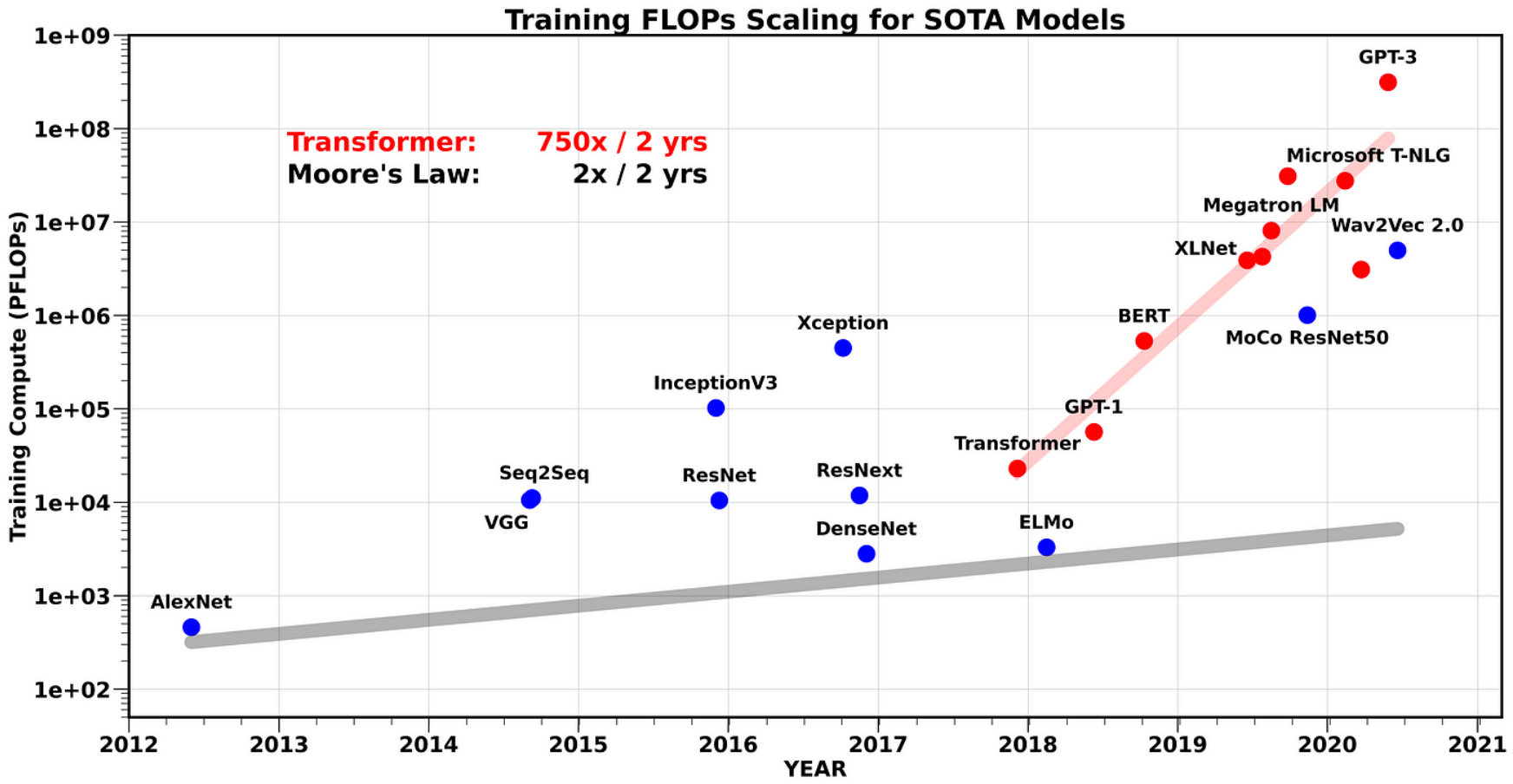
- 2006: IBM Blue Gene L ≈300TFLOPS



- 2024: Nvidia RTX5090 > 200 TFLOPS



Model Size & Computing Power



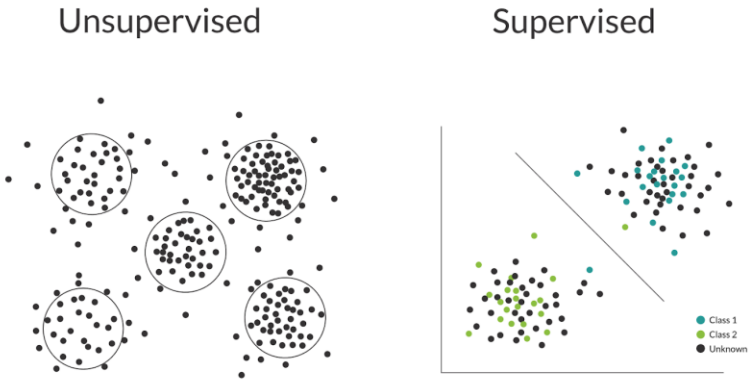
Content – Block II

- Supervised, Unsupervised Learning
- Loss, Quality, Accuracy
- What are Labels?
- Training, Validation, Testing

- Datasets
- Bias – Variance Trade-off
- IID – Independent, Identically Distributed (IID)

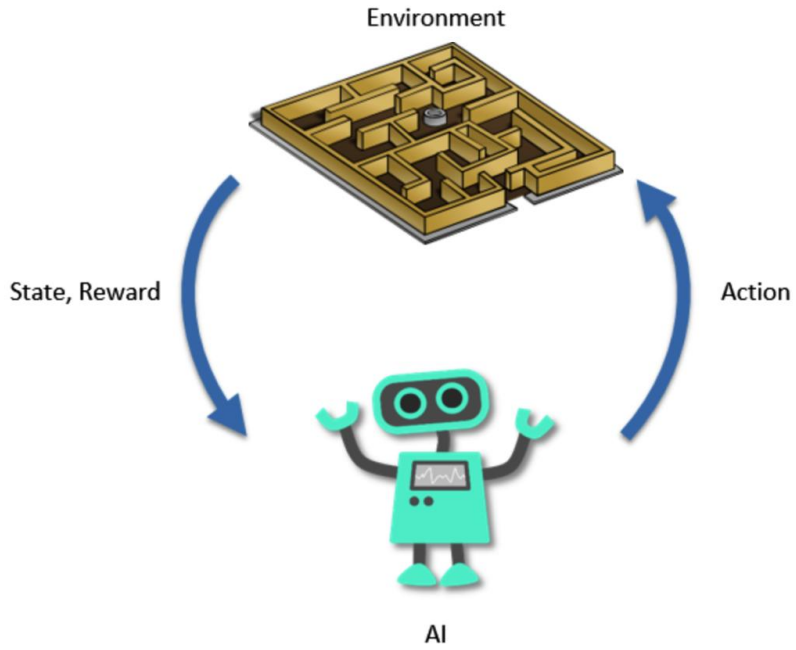
Supervised vs Unsupervised Learning

- **Supervised Learning:**
  - Making predictions based on data.
  - Training data with corresponding labels.
  - Model accuracy can be directly determined.
- **Unsupervised Learning:**
  - Understanding of data.
  - Searching for structure or patterns, correlations.
  - Not specific to a label (unsupervised).
  - Does not necessarily require labels.
  - Evaluation is often indirect or only qualitative.



Reinforcement Learning

- **Reinforcement Learning:**
  - Targeted Learning
  - Reward if the Agent gets better.
  - Games, Model Predictive Control



- Example: Gran Turismo Sophy

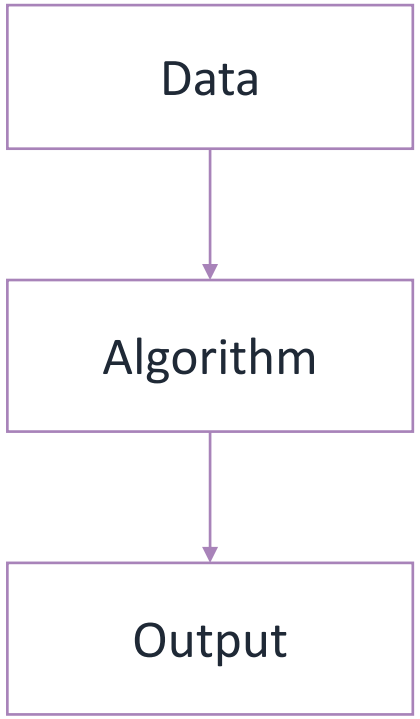
https://www.youtube.com/watch?v=AmtzhWilq0I

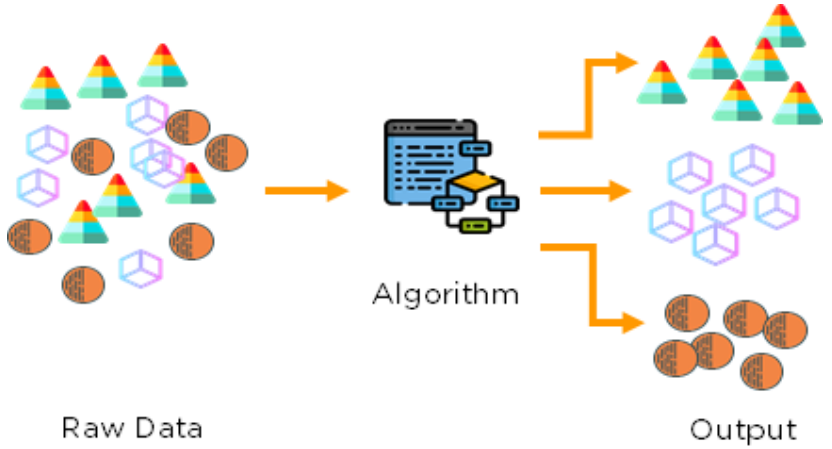
Supervised and Unsupervised, that’s it?

- Semi-Supervised:
  - Often used to improve supervised algorithms.
  - Reality: Many data points, but few have labels.
  - Example: Network Intrusion Detection.

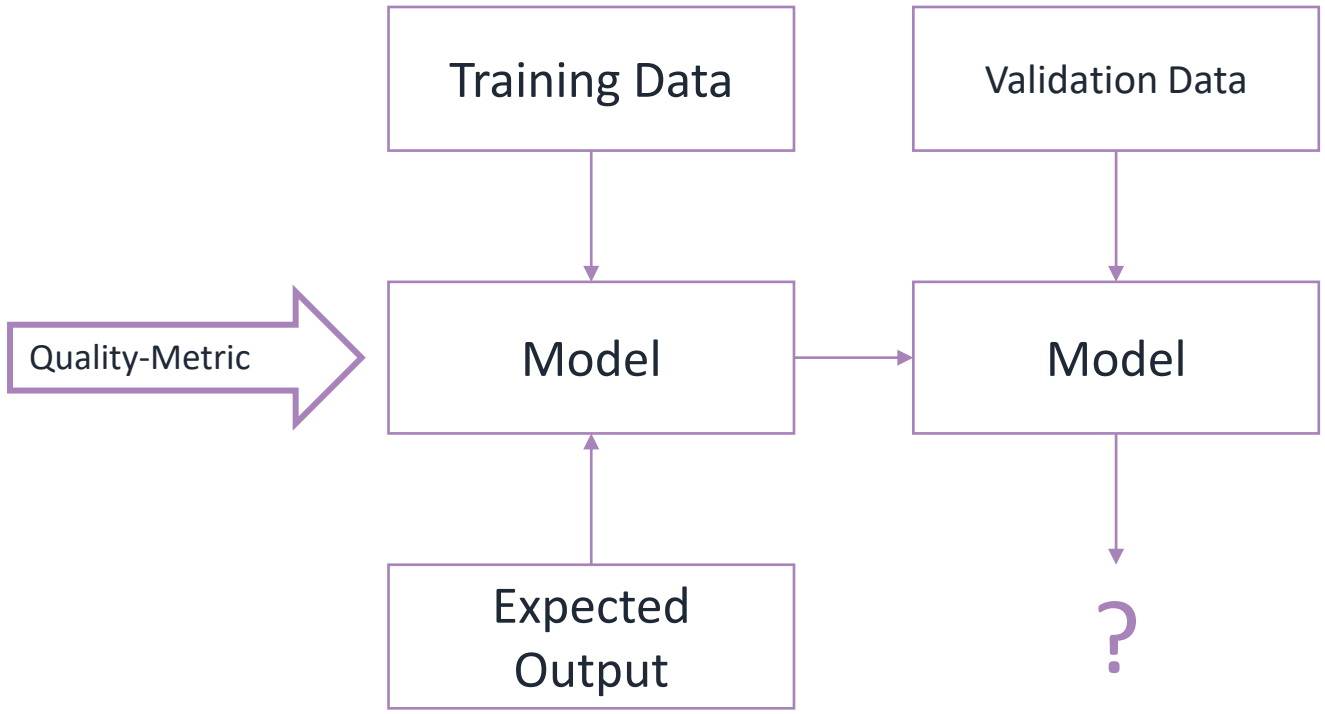
- Advantages:
  - Requires fewer experts for dataset creation.
  - Offers significantly more possibilities than unsupervised learning.

Unsupervised Models

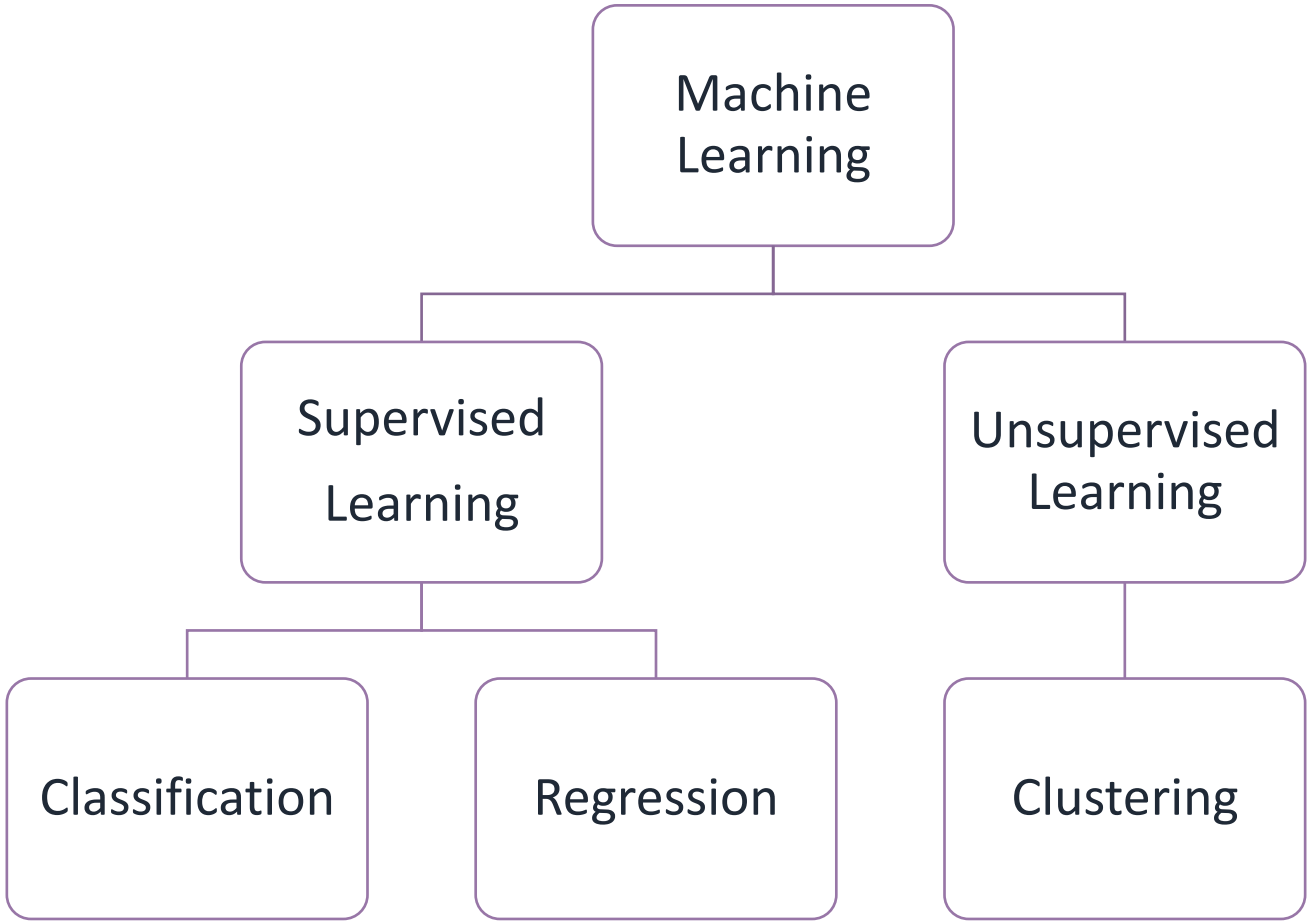




Supervised Models



Methods and Propierties



Overview on Methods

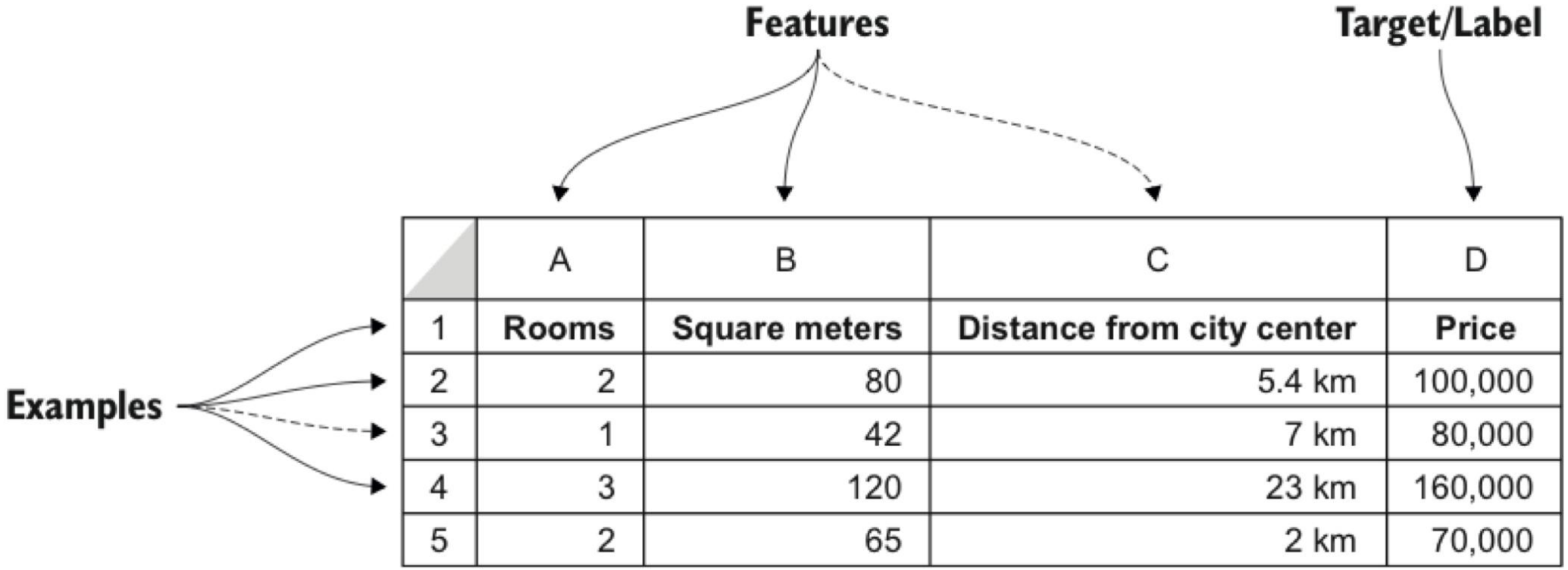
- Supervised:
  - Regression (Really?)
  - SVM (Support Vector Machine)
  - Decision Trees
  - Neuronal Networks: Problem -> Label
- Unsupervised:
  - Clustering
  - Neuronal Networks Problem -> Problem (Transformer)
- Semi-Supervised:
  - Neuronal Networks: Problem -> Problem & Problem -> Label

Let’s talk about Features?

- A Feature is a ‘Single Datapoint’: **X, y**
  - X and y is driven by Math and Stats.

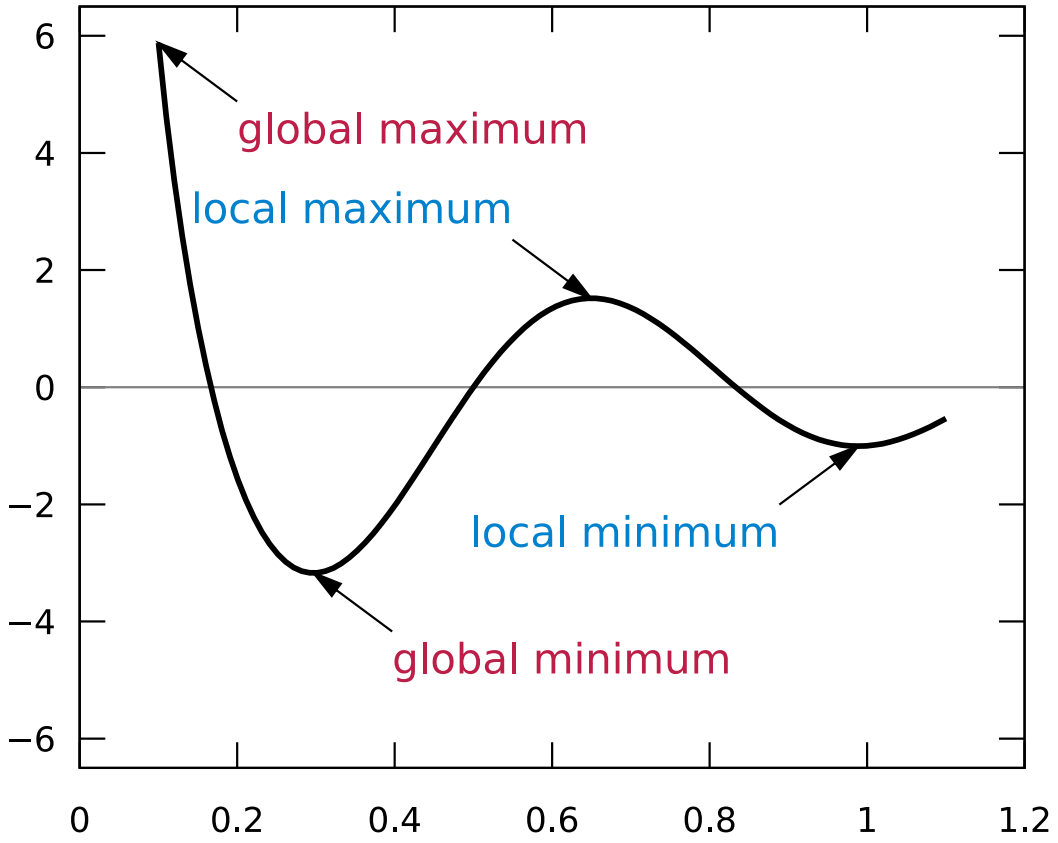
- Numerical Values can stay numerical: 27 is 27!
- Categorical Values can be handled like Enums:
  - Car: 1, Bike 2
- Sometimes abstract representations are necessary:
  - Cat is at home
  - Dog is at home
  - Represented by 2 Vectors

Features, Examples, Labels

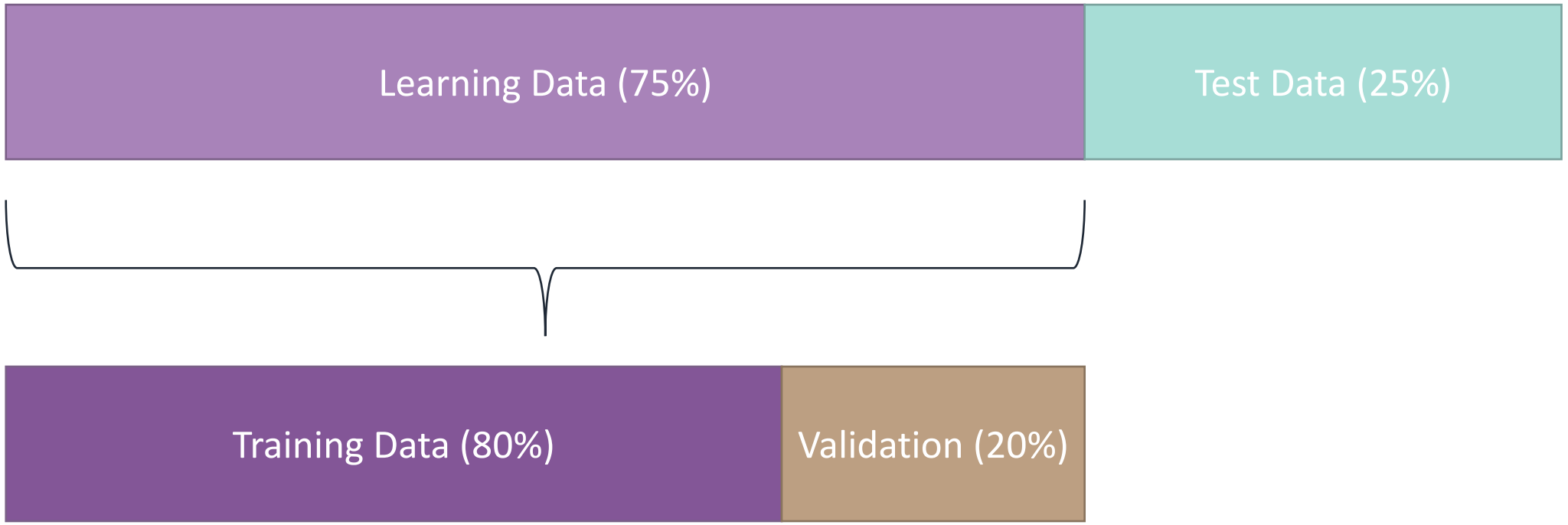


	A	B	C	D
1	**Rooms**	**Square meters**	**Distance from city center**	**Price**
2	2	80	5.4 km	100,000
3	1	42	7 km	80,000
4	3	120	23 km	160,000
5	2	65	2 km	70,000

Supervised Learning: Optimization Problem



Data Distribution



Training, Validation & Testing

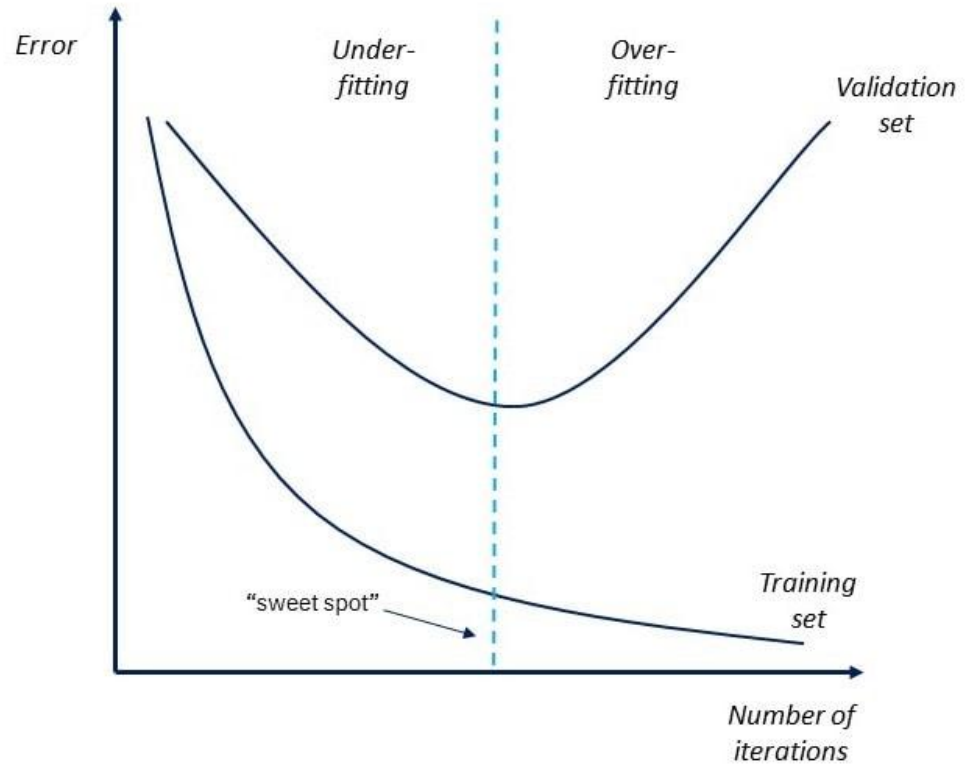
- Training Set:
  - Used to train the model.
  - These data can be manipulated as needed.
- Validation Set:
  - Validation of the trained model.
  - Checking the hyperparameters.
  - How well has the learner (model) performed?
- Test Set:
  - Independent dataset used solely for quality control.
  - Data distribution must be analogous to the training set and validation set (e.g., 10 fruits, 3 apples).

Hyperparameter?

- Hyperparameters are control settings that influence the model.
  - Decisions like which model to train (regression, forest, SVM, etc.) are made through hyperparameters.
  - Hyperparameters are not learned during training; they determine the model's architecture.
  - In function fitting, considerations include the degree (2nd, 3rd, 4th, etc.) such as x^2, x^3, x^4.

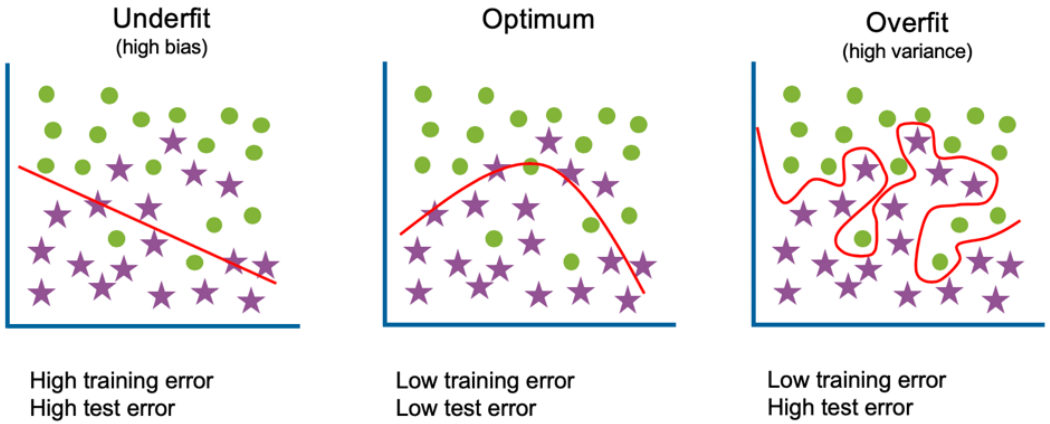
- Hyperparameter tuning is often performed on the training and validation sets.
  - Grid Walk
  - **Bias**: "He who looks at the validation set too often knows too much."

Overfitting & Underfitting



- Underfitting:
  - Training and validation 'in-sync' but performing poorly!
  - The problem may be too 'difficult' for the model.
  - Requires a more sophisticated model.
  - More 'training time' may be necessary.
  - Consider incorporating more features or gathering more data.
- Overfitting:
  - Training great – validation bad
  - More Data
  - Model Complexity to high

Bias Variance – Trade-off



- Underfit
  - To “simple”
- Optimum:
  - That’s what we need!
- Overfit:
  - Bad generalization

Question

Why does the training error decrease monotonically with increasing model complexity?

Question

Why does the training error decrease monotonically with increasing model complexity?

- The larger model allows for a larger search space.
- More parameters enable 'more learning

Question

Why is the Test Error decreasing with increasing model complexity?

Question

Why is the Test Error decreasing with increasing model complexity?

- As long as we ‘under-fit’, more information can be ‘learned’. Training and Test errors decrease.
- Higher model-complexity enables more «insight»

Question

Explain what is happening after the optimum, and why?

Question

Explain what is happening after the optimum, and why?

- Over-fitting
- We learn »noise» instead of «signal».

Question

Why is the Bias-Variance trade-off so important?

Question

Why is the Bias-Variance trade-off so important?

- It’s the fundamental trade-off of too much, and too little.
- We know: If it looks too good, something is fishy.

Recap: Supervised Learning Notation

- X: Training Data
- y: Training Label
- $\hat{y}$: Predicted Label

- $\tilde{X}$: Test Data
- $\tilde{y}$: Test Labels

Recap: Supervised Learning Notation

X

Eggs	Milk	Fish	Soja	Nuts	Gluten
0	0.7	0	0.3	0	0
0.3	0.7	0	0.6	0	0.01
0	0	0	0.8	0	0
0	0.7	1.2	0	0.1	0.01
0.3	0	1.2	0.3	0.1	0.01

Sick
1
1
0
1
1

y

- Feature Matrix X: Columns correspond to features, Rows correspond to Data Sample
  - X_ij = Amount of Food j on Day i
  - X_i = Amount of Food in total on Day i
- Label y: Contains whether ”sick” or “not sick”.

Is it even possible to learn?

- Does our Training Error tell us something about our Test Error?
  - **HELL NO:** Test-Data might have nothing to do with Training-Data
  - Changing of circumstances (Lactose Tablets, Side-effects of other meds)

X

Eggs	Milk	Fish
0	0.7	0
0.3	0.7	0
0	0	0

Sick
1
1
0

y

$\tilde{X}$

Eggs	Milk	Fish
0	0.7	0
0.3	0.7	0
0	0	0

Sick
0
0
0

$\tilde{y}$

IID Assumption

- In order to ”learn”, we have to straighten a few things:
  - Training-Data, Validation Data and Test-Data need the same sampling context!

- **Independent and identically distributed (IID)**
  - One dataset split into a training, validation and test set
  - Any occurrence has the same likelihood independent of training or test set

Dataset Splitting:

- In principle, the training, validation, and test sets must have the same distribution!

- Out of 100 features: x for training, y for validation, z for testing
  - x, y, z are "consistent within themselves"!

- At the same time, we consider:
  - Do we have "uneven" distributions?
  - Are we temporally constrained? (e.g., stock prices, factories)

IID Assumption

- **Independent and identically distributed (IID):**

- The IID assumption is satisfied when:
  - All examples have/come from the same distribution.
  - The examples are independent (the order does not matter).
- Example:
  - The training set, validation set, and test contain blue and red balls.
  - The probability P_blue is the same in set train, val and test!

IID Assumption: Kartenspiel



- Französisches Blatt:
  - 4 Farben (Kreuz, Pik, Herz, Karo)
  - 9 Wertigkeiten (6-10, Bube, Dame, König, Ass)

- Ist IID Assumption erfüllt?
  - Ziehen einer Karte (oberste), zurücklegen, mischen, wiederholen
  - Ziehen einer Karte, zurücklegen, wiederholen
  - Ziehen einer Karte, *nicht* zurücklegen, mischen, wiederholen

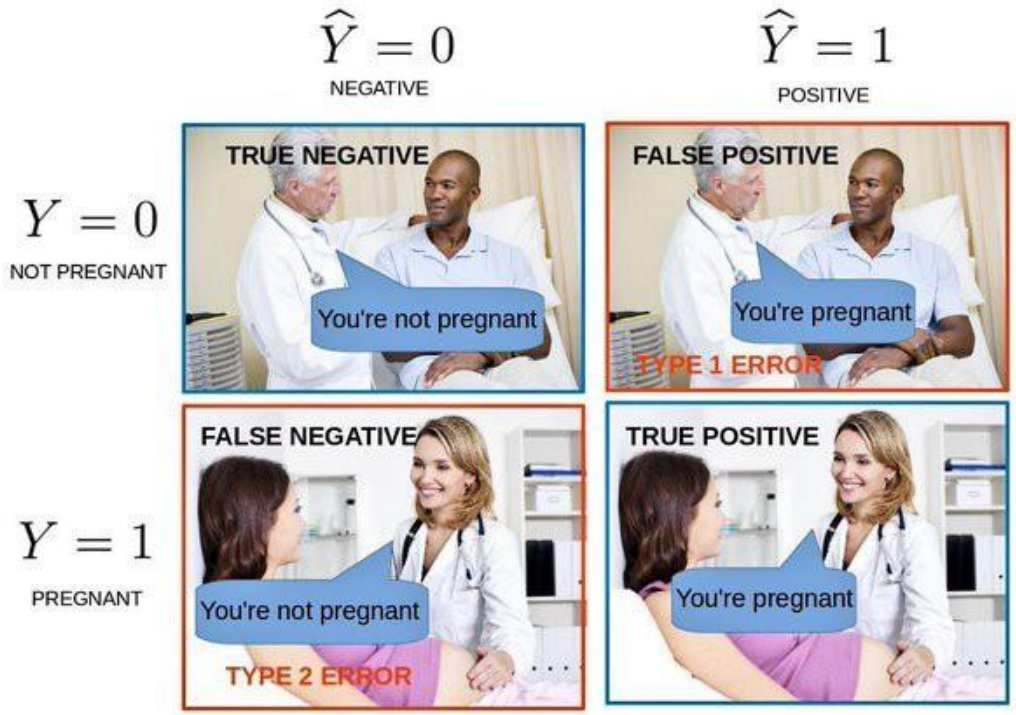
Optimisation Bias

- Nehmen wir einen multipe-choice Test (a,b,c,d) mit 10 Fragen:
  - Zufälllig ausgefüllt: 25% richtig
  - Zufällig 2x ausgefüllt und den Besseren gezogen: 33% richtig
- Skalieren wir das Experiment auf 10 Tests: 47%
  - Bei 100: 62%
  - Bei 1000: 73%
- Wenn man viele Versuche hat, dann hat man die Möglichkeit es gut zu tun.
- Bester Score mit *Random.seed(42)*!

Metriken

- Confusion Matrix für 2-Klassen Klassifikation
- Metriken: Accuracy, Recall, Precision, F1-Score
- Unterschied zwischen Recall und Precision
- Decision Threshold: ROC Kurve und AOC Fläche

Confusion Matrix

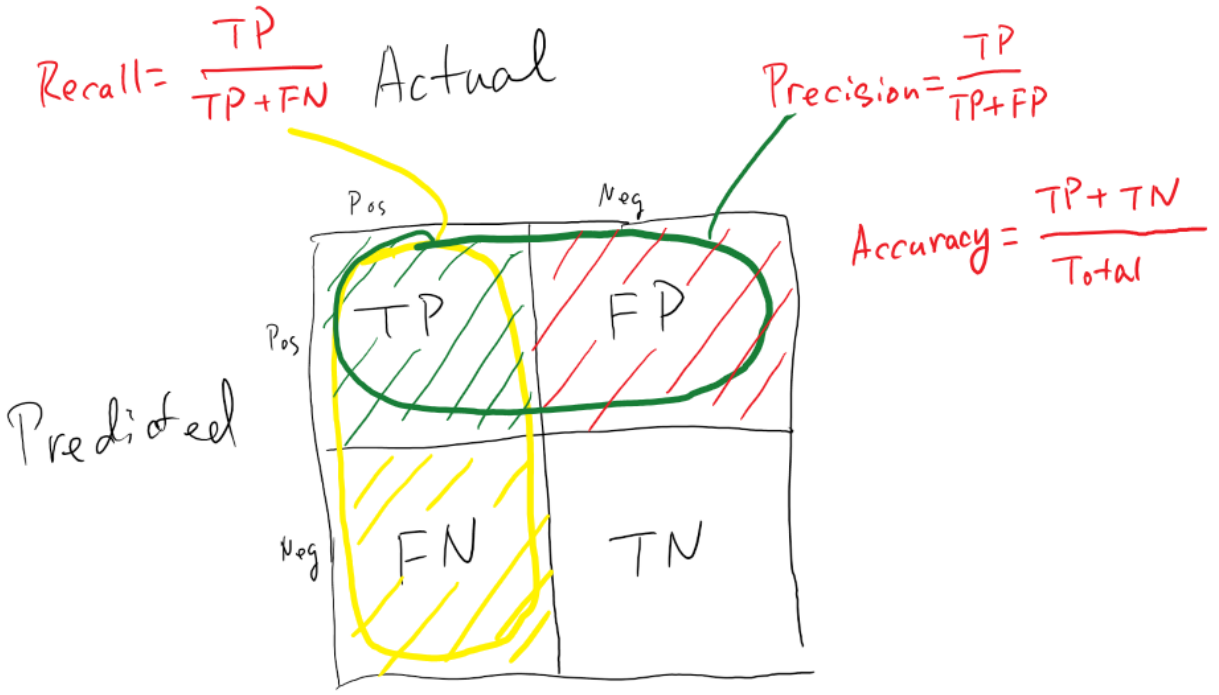


		Echte Werte (Actual Values)	
		1	0
Vorhergesagte Werte (Predicted Values)	1	True Positive (TP)	False Positive (FP): Type 1 Error
	0	False Negatives (FN): Type 2 Error	True Negatives (TN)

Metriken

- $Accuracy = \frac{Correct}{Total} = \frac{TP+TN}{TP+TN+FP+FN}$
- Recall: ‘Wieviele relevante Daten habe ich erwischt?’
  - $Recall = \frac{TP}{TP+FN}$
- Precision: ‘Wieviele ausgewählte Daten sind relevant?’
  - $Precision = \frac{TP}{TP+FP}$
- $F1 - Score = \frac{2}{\frac{1}{Recall}+\frac{1}{Precision}} = 2 * \frac{Precision*Recall}{Precision+Recall}$

Precision und Recall



Case 1: Covid

- Kosten für FN > Kosten für FP

- Labels:
  - Covid-Positive: 1
  - Gesund: 0

		Echte Werte (Actual Values)	
		1 (hat Covid)	0 (gesund)
Vorhergesagte Werte (Predicted Values)	1	True Positive (TP)	False Positive (FP): Type 1 Error
	0	False Negatives (FN): Type 2 Error	True Negatives (TN)

Hat Covid, aber Diagnose sagt gesund!

Case 1: Covid

- TP sind Covid Patienten mit Covid Diagnose
- TN sind gesunde Patienten mit gesunder Diagnose

- FP: Schade 10 Tage Quarantäne für nichts!
- FN: Potentielles *Risiko*, da Ansteckungsmöglichkeiten!

- FN ist für das System teurer als FP: ***Recall***

Case 2: Spam

- Kosten für FN < Kosten für FP

- Labels:
  - Spam: 1
  - Kein Spam: 0

		Echte Werte (Actual Values)	
		1 (Spam)	0 (not Spam)
Vorhergesagte Werte (Predicted Values)	1	True Positive (TP)	False Positive (FP): Type 1 Error
	0	False Negatives (FN): Type 2 Error	True Negatives (TN)

Das Jobangebot von Google

Case 2: Spam

- TP sind Spam Mails, welche als Spam Mails erkennt werden
- TN sind legitime Mails, welche als solche erkennt warden

- FP: Das Job Angebot von Google
- FN: Der Persische-Prinz und sein Milionen-Erbe

- FP hat weittragendere Konsequenzen als FN: ***Precision***

Exercise: Credit Scoring

- Time: 15 minutes

- You are responsible for building a model that decides whether a loan is worth granting or not.
  - Label 1: Bad Load (Bank loses money, Loan doesn’t get repaid)
  - Label 0: Great Credit (Bank makes money, no bureaucracy).

Afterthought: F1-Score vs Accuracy

- Accuracy: Focus on TP (True Positives) and TN (True Negatives)
  F1: Focus on FP (False Positives) and FN (False Negatives)

- Accuracy is ‘intuitive’ - while F1 is somewhat more "artificial»
- Accuracy works well when classes are balanced.
- However, the F1 score also works when there is class imbalance.

- In reality: F1-score is more useful than accuracy:
  - I’ll admit it: My primary focus is on accuracy and confusion matrix.

Label Prediction is Probabilistic!

- Labels are probabilistic predictions (stochastic ML Models)
  - The values range between [0;1]
  - The closer the prediction is to 0 – the more likely it is 0.
  - The closer the prediction is to 1 – the more likely it is 1.

- Great Model: A great model has a sharp separation.
- Apply Threshold-Tuning to fit your business case!

Warum Neuronale Netzwerke?

- Popularität in Hype-Wellen:
  - Zur Zeit: Eines der heissten Themen in der Wissenschaft

- Aktuelles Hoch getrieben durch unbeschreibliche Performance in:
  - Computer Vision Tasks (Bild Erkennung, Semantische Segmentierung)
  - Language Recognition (Speech 2 Text)
  - Language Models (Sprach Verständnis, Interpretation)

Warum Neuronale Netzwerke II?

- Haupt-Treiber für den Erfolg:
  - Grosse qualitativ hochwertige Datensets: 100TB+
  - Sehr tiefe Netze (Deep Learning)
  - Unglaubliche Computerleistung zu ‘keinem’ Geld.

- Einige Tweaks an der Struktur der Netzwerke
  - CNN (Convolution)
  - LSTM (Long—Short Term Memory)

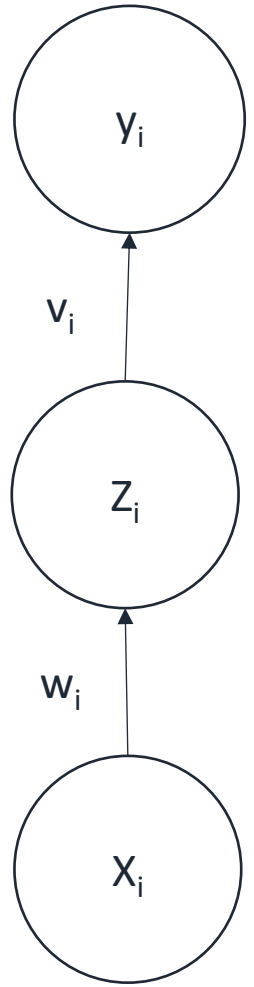
- Mathematik praktisch unverändert seit 1960…

Neural Networks: Motivation

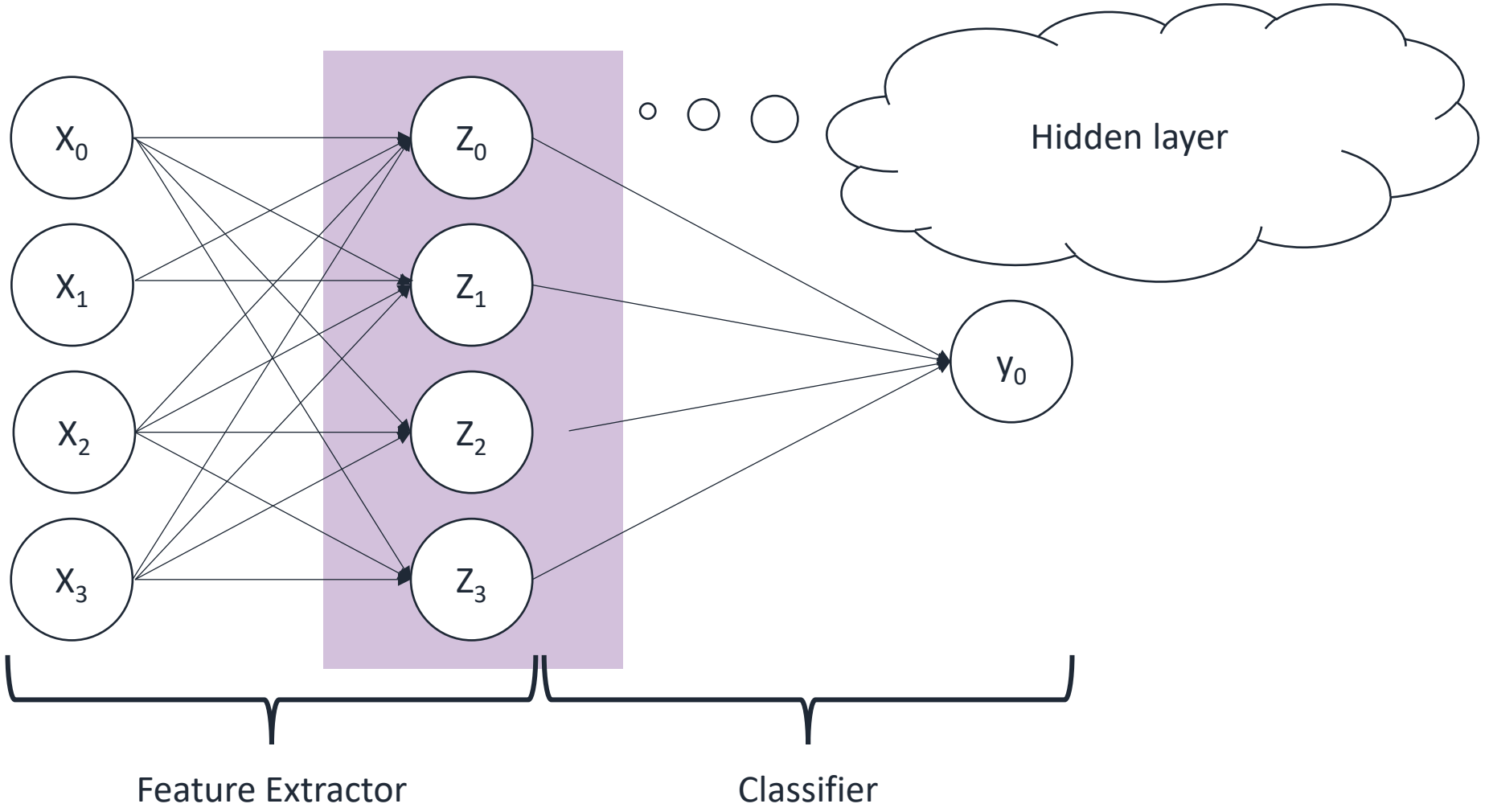
- Viele Domänen brauchen eine nicht lineare Transformation der Features
  - Aber manchmal ist es offensichtlich
    - Bsp. SciKit learn
- Neuronale Netzwerke versuchen diese Transformation zu lernen
  - Optimierung der ‘Feature’ für bestes Resultat
- Wir fangen mit einem hidden-layer an und gehen dann Deep…

Supervised Learning Roadmap

- **Level 1:** «Direct» Supervised Learning
  - Wir lernen Parameter *w* basierend auf x_i und y_i.
- **Level 2:** Basis Transformation
  - Wir lernen Parameter *v* basierend auf Basis Z_i und y_i. (x_i -> z_i existiert)
- **Level 3:** PCA, t-SNE, Latent-Factor Models (Manifolds)
  - Wir können Parameter *w* von Basis Z_i aufgrund von X_i erlernen
  - Wir können Parameter *v* für Z_i -> y_i lernen. (Classification)
- **Level 5:** Neuronale Netzwerke
  - *w* und *v* werden direkt basierend auf X_i und y_i gelernt
  - Basis Z_i wird impliziert gelernt, welche gut geeignet für Supervised Learning Classification ist.

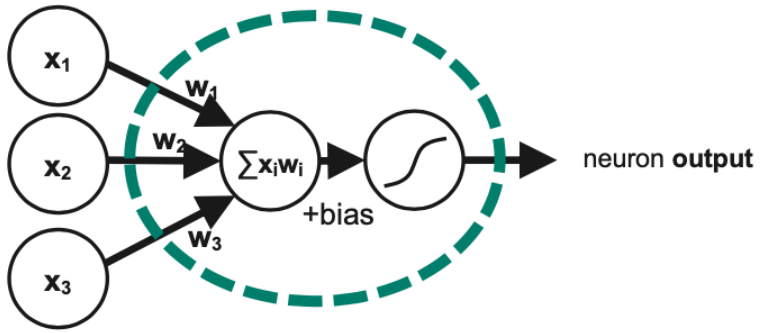


Feature Extractor & Classifier

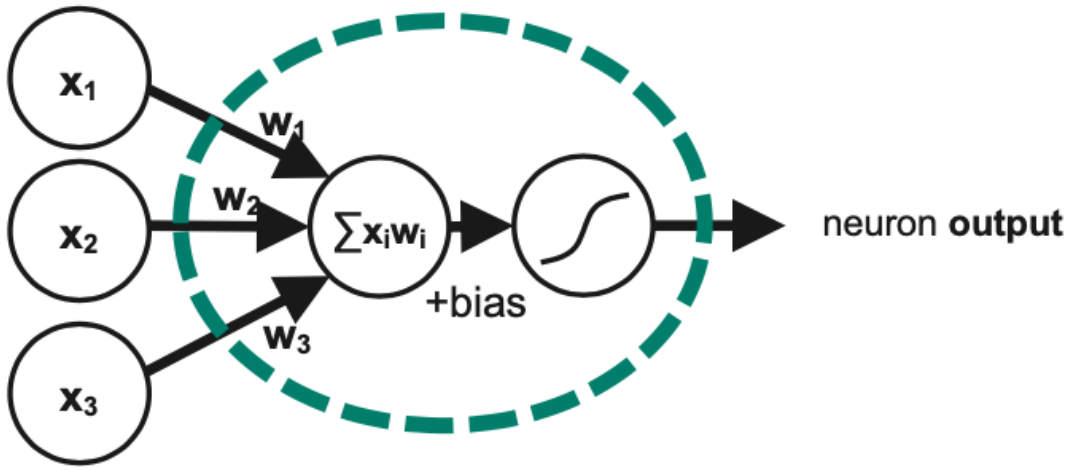


Neuronales Netz als Funktion

- Das Neuron erhält Input [x_1, x_2, …]
- Jedes Neuron besitzt eigene Eingangsgewichte *W*, einen Bias *b*.
- Output entspricht der Summe der gewichteten Inputs



Neuronales Netz als Funktion II

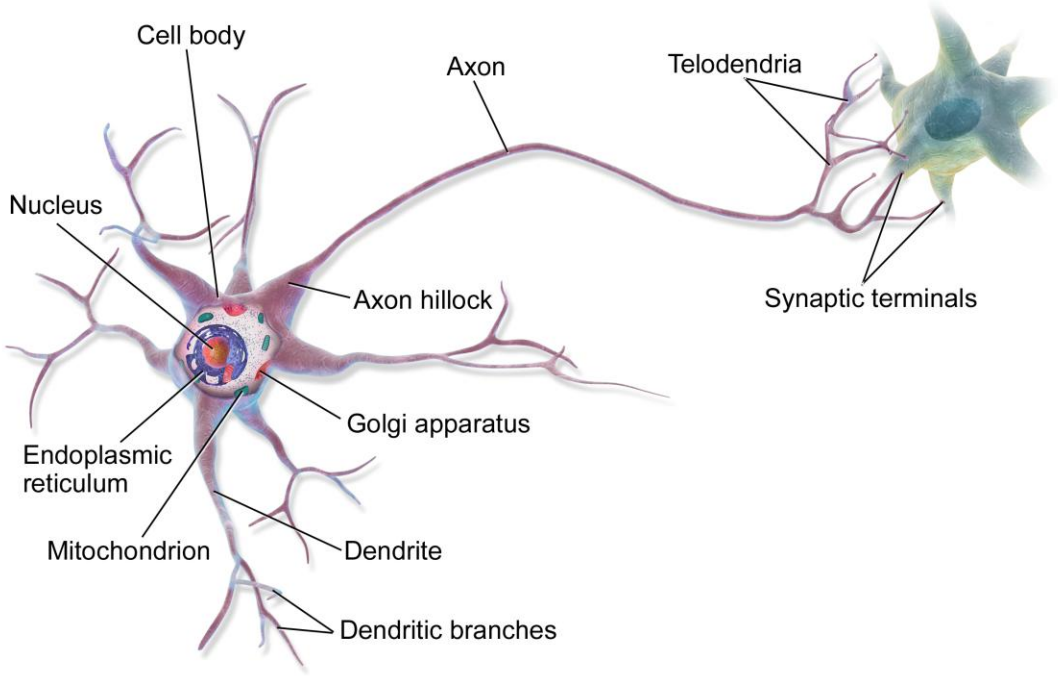


$$\sigma\left(\sum_i w_i f_i + b\right)$$

Aktivierungsfunktion?

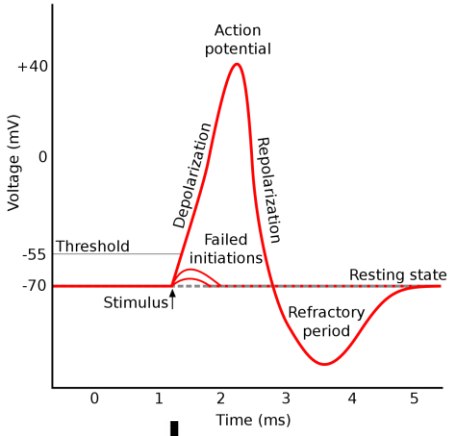
- Avoid Linear Combinations:
  - A*x + b*x = (a+b)*x

Biologische Sicht

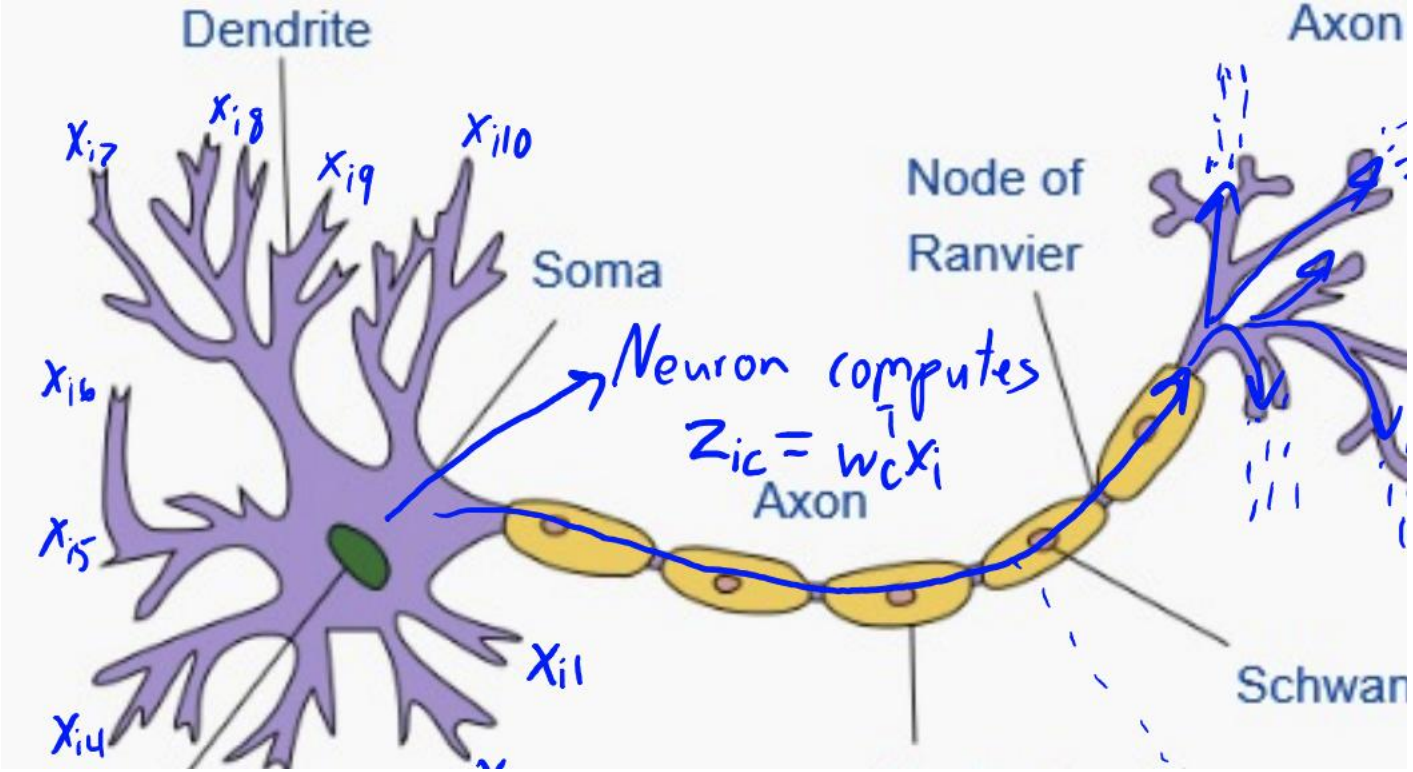


https://en.wikipedia.org/wiki/Neuron

- Neuron hat ‘Dendriten’ als Inputs
- Neuron hat ‘Axon’ als einzelnen Output
- Mit dem richtigen Input, wird ein Output getriggert

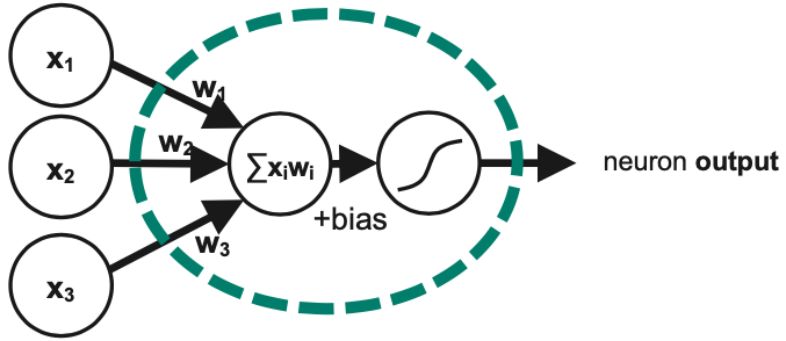


Biologische Sicht II



Single Layer Perceptron

- Fähigkeit linear separierbare Muster zu lernen
- Hohe Input Dimensionalität -> lineare Separierbarkeit, aber im n-dimensionalen Raum
- Terminologie:
  - Perceptron ist ein ‘Einzelnes Neuron’
  - Perceptron ist aber auch der Algorithmus zum trainieren eines binären Klassifizierers (Classifier-part)



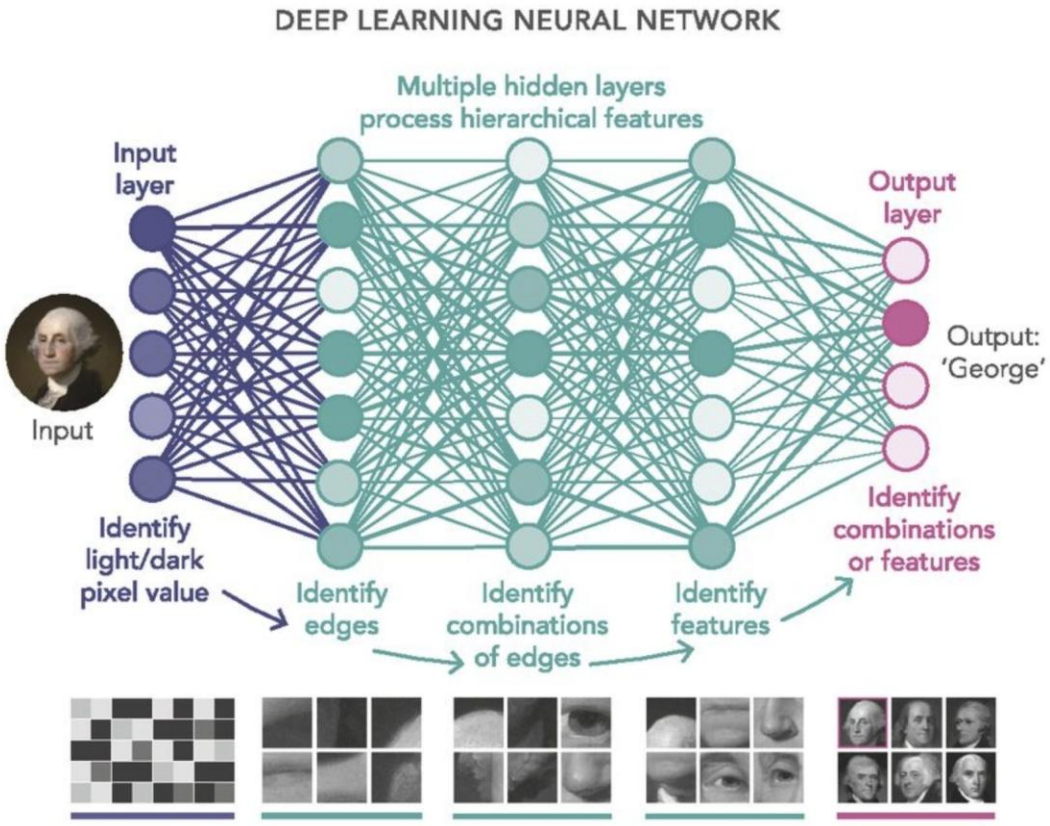
- Layer können geschichtet (stacked)werden. Diese Architektur nennt man Multi Layer Perceptron MLP.
- Lasagne war eines der ersteren Deep Learning Frameworks! ☺

Stacking Layers -> Deep Learning

- Der Output des jeweiligen Layers, entspricht dem gewichteten Input des nächsten.
  - Komplexe Features kombinieren und aggregieren Informationen durch die Netztiefe
  - Alle Parameter werden mithilfe des Backpropagation Algorithmus erlernt. (Vorlesung Oliver am Donnerstag).
- Aktuelle KI ist angetrieben durch tiefe Modelle, mit Miliarden von Parametern (auch GPTv4!)
- Die freien Parameter werden durch **UNMENGEN** an Daten ‘trainiert’.

Visualisierung: Tiefe Neuronale Netzwerke

- Tiefe neuronale Netzwerke werden auch Deep Neural Networks genannt
- ANNs lernen Features zu erkennen und kombinieren in tieferen Schichten diese zu komplexeren Mustern
- Das Resultat ist nichts anderes als eine komplexe mathematische Funktion welche X_i auf y_i abbildet, bestehend aus vielen einfachen Komponenten.



https://www.positronic.ai/consulting/deep-learning/

Kleines Gedankenexperiment

- Schauen Sie sich die vorherige Folie an:
  - Wie würden Sie das Netz implementieren?
  - Besprechen Sie die Konzepte mit der Person neben ihnen.

- Implementieren Sie nicht!

Künstliche NN vs Reale NN

- Künstliche NN:
  - X_i ist Feature Darstellung der Welt
  - Z_i ist interne Darstellung der Welt
  - Y_i ist Output / Klassifikation o. Regression
- Reale NN:
  - Timing / Zeit-diskrete Aktion sind wichtig (Kontinuierliches Signal)
  - Hirn ist ‘höchst organisiert’ (Bsp. Strukturen für diverse Tasks)
  - Verbindungen -> Struktur Änderungen
  - Unterschiedliche Neurotransmitter (Elektrochemisch, Hormonell, etc.)

Agenda

- Intro classical ML Methods
- What is it
- Terminology
  - Models, Data, Cost Function
- Popular Examples
  - Decision Tree
  - KNN
- Lab: Get your hands dirty

VeeMG




Diet
Coke
Sugar-free cola
WITH ASPARTAME AND ACESULFAME-POTASSIUM
0
CALORIES PER CAN
355 mL
330ml
Energy 5kJ 1kcal
Fat 0g
Saturates 0g
Sugars 0g
Salt 0g
0%* 0%* 0%* 0%* 0%*
100ml: 1.6kJ/0.4kcal

What is a model

- Function approximation and optimization

- Assumption:
  - There exists a function / relationship / set of rules
  - To Interpret data in a certain way
  - **y = g(x) is the «given» function**

- Approximation
  - **y^ = f(X, p)**

Model vs Algorithm

- An **Algorithm** is used to train a **Model**

- We can think of the **Algorithm** as a function
  - We give the algorithm data and it produces a model
  - **Model** = **Algorithm**(Data)

- Models are the specific representations learned from data
  - Example
    - f(x) = ax+b is an **Algorithm**
    - f(x) = 4x + 2 is a **Model**

Classification Problem

- A **classification problem** is when you have a set of data and your goal is to **assign each data point to a specific category**

- **Email Classification**
  - Predict if an email is "Spam" or "Not”
- **Medical Diagnosis**
  - Predict if a patient has a certain disease (like "Diabetes" or "No Diabetes") based on medical test results.
- **Image Recognition**
  - Classify images as "Dog," "Cat," or "Bird"

Apple & Pear Classification

- Let’s design an algorithm to differentiate between apples and pears given their width and height

- Question:
  Given width and height what is the correct fruit class

- What would be your approach as a programmer?

Width	Height	Fruit
7.1	7.3	Apple
7.9	7.5	Apple
7.4	7.0	Apple
8.2	7.3	Apple
7.6	6.9	Apple
7.8	8.0	Apple
7.0	7.5	Pear
7.1	7.9	Pear
6.8	8.0	Pear
6.6	7.7	Pear
7.3	8.2	Pear
7.2	7.9	Pear

Decision Tree

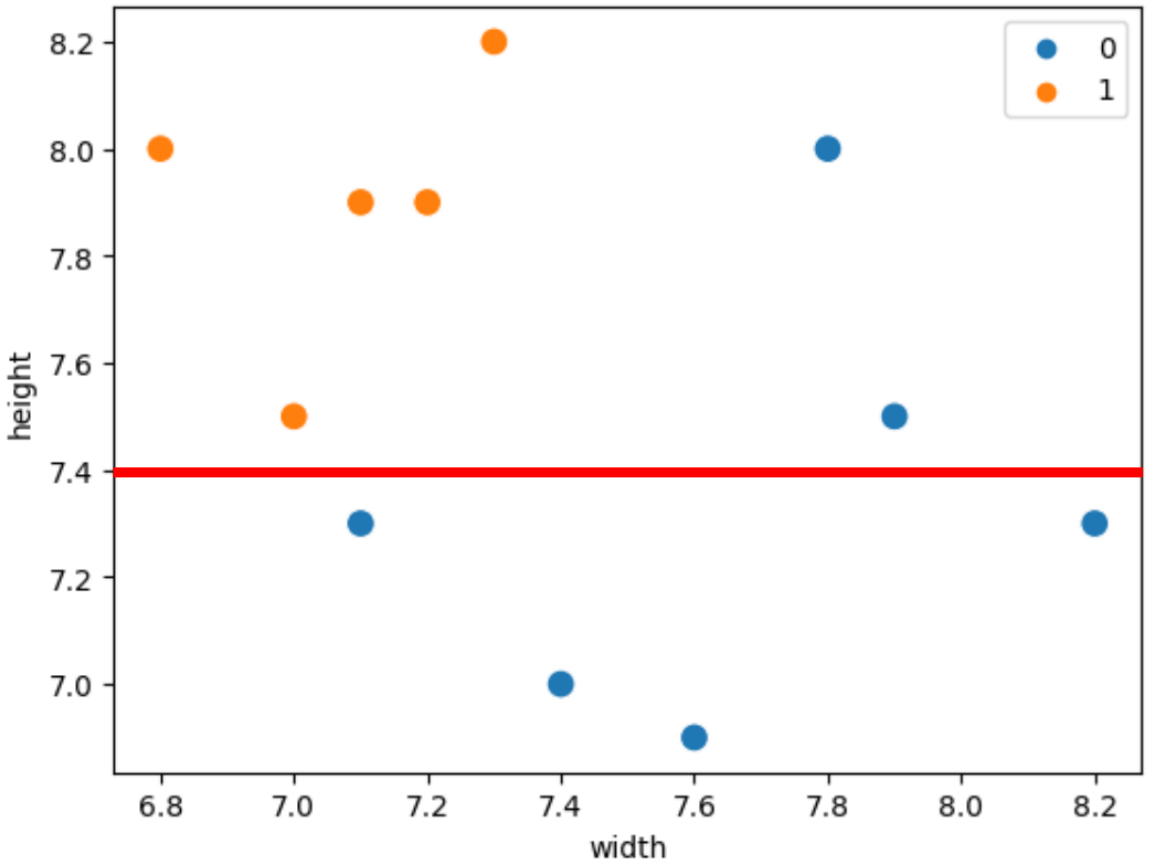
- Simple if else structure

```
def get_fruit_class(width, height):
    if width < 7.35:
        if height < 7.4:
            return 'Apple'
        return 'Pear'
    return 'Apple'
```

Width	Height	Fruit
7.1	7.3	Apple
7.9	7.5	Apple
7.4	7.0	Apple
8.2	7.3	Apple
7.6	6.9	Apple
7.8	8.0	Apple
7.0	7.5	Pear
7.1	7.9	Pear
6.8	8.0	Pear
6.6	7.7	Pear
7.3	8.2	Pear
7.2	7.9	Pear

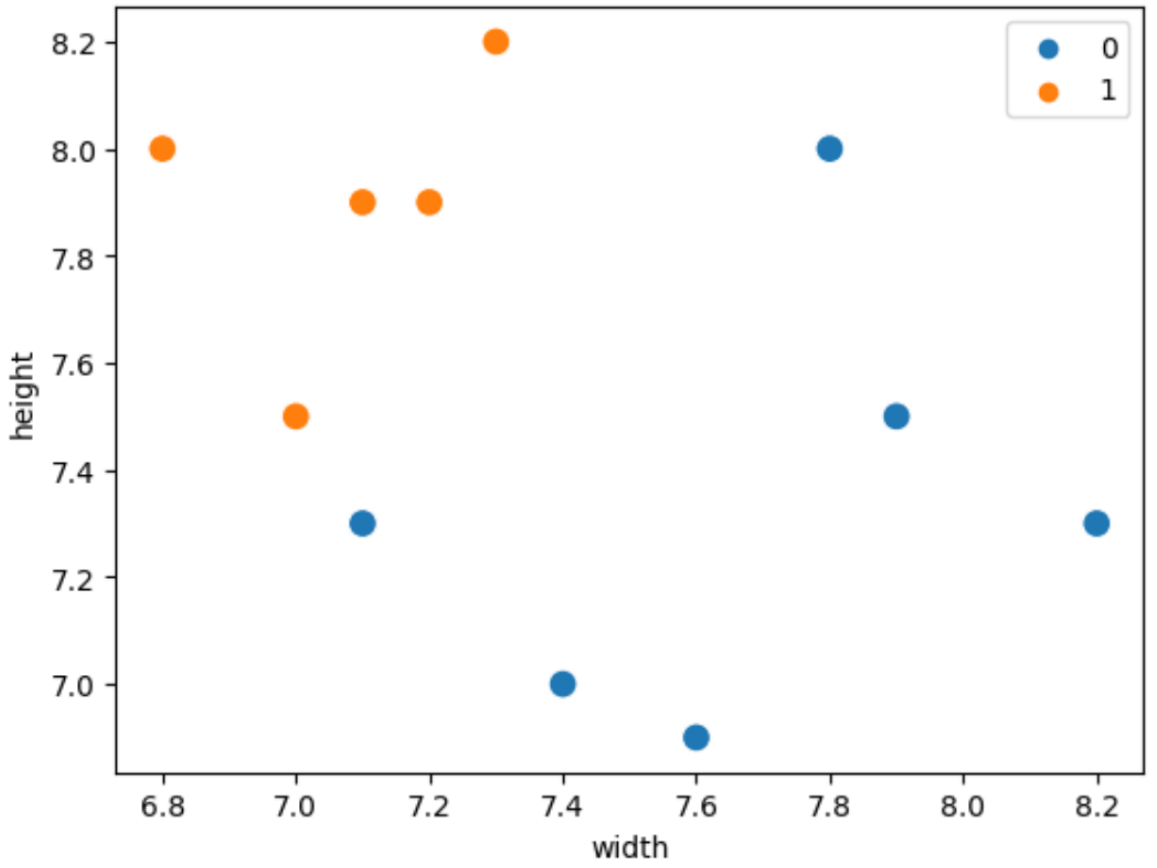
Decision Tree

- Only use vertical or horizontal lines to create homogeneous splits
- How does a programmer find that line?



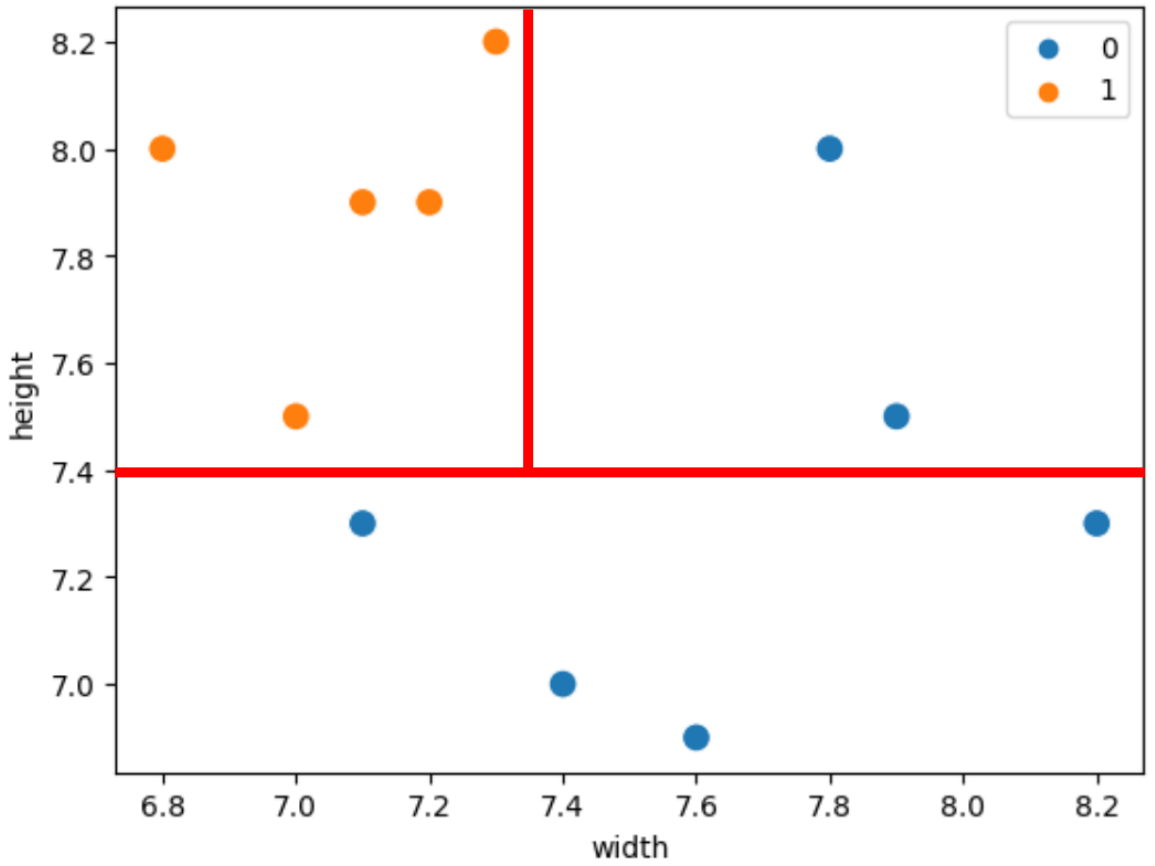
Decision Tree Idea

- Create **splits** by recursively dividing regions into subsplits
- Finish if split is **homogeneous**
- A **homogeneous split:** only contains datapoints from same class



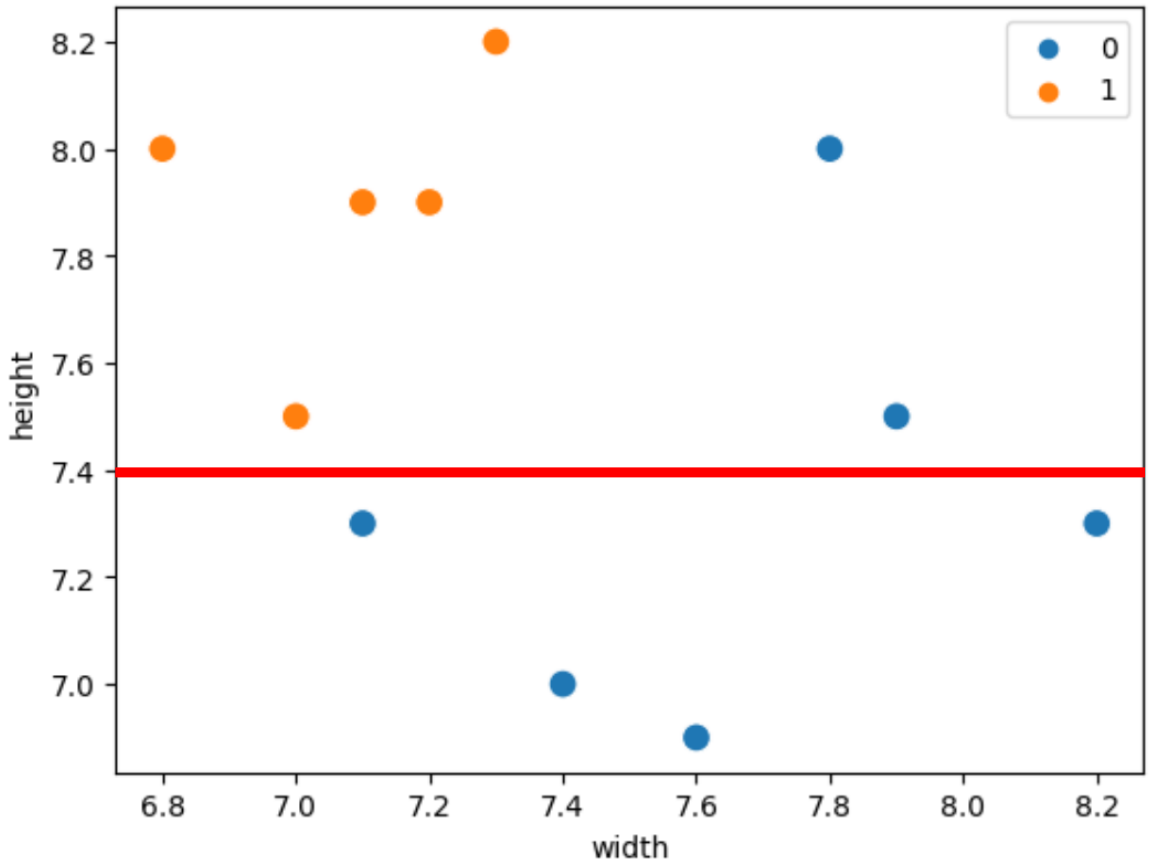
Decision Tree

- Divide splits further recursively



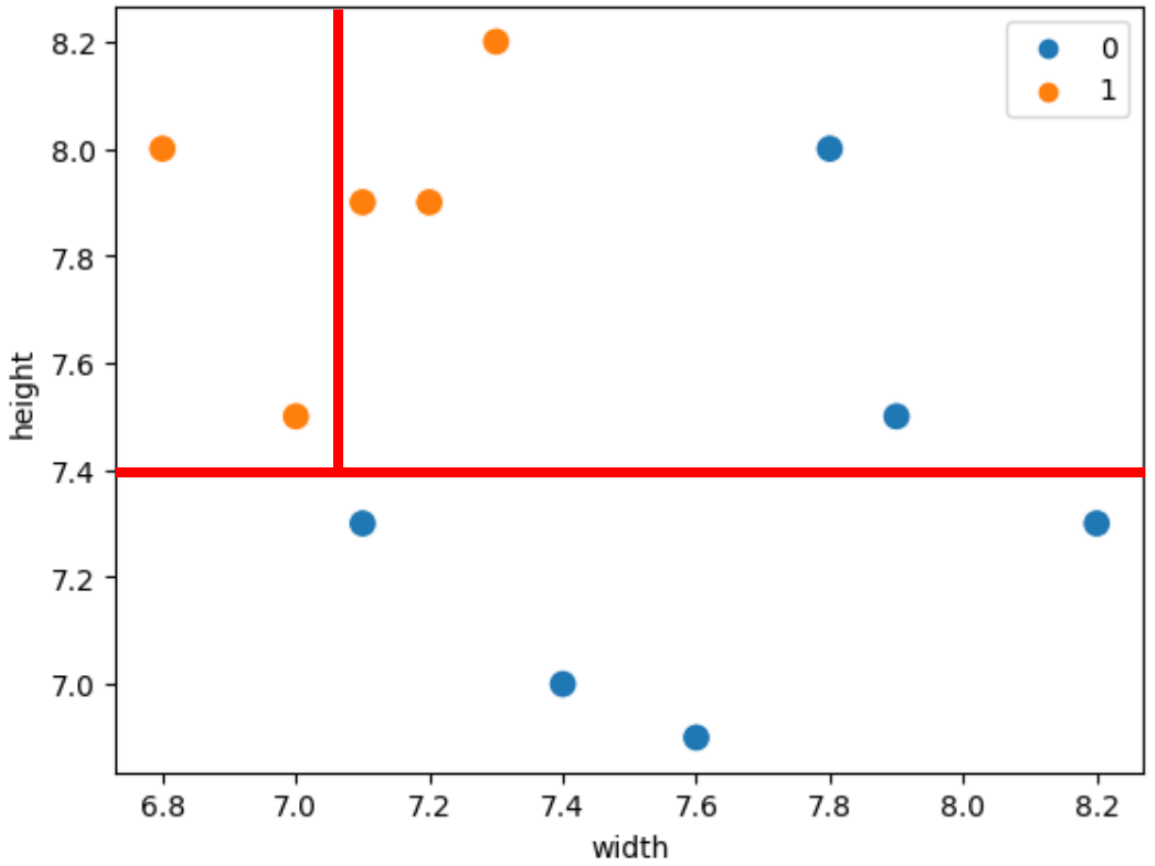
Decision Tree

- Wait what is a good split?
- Define quality for split
  - Ideas?
  - GINI
  - Entropy
  - Concept of «Information Gain»



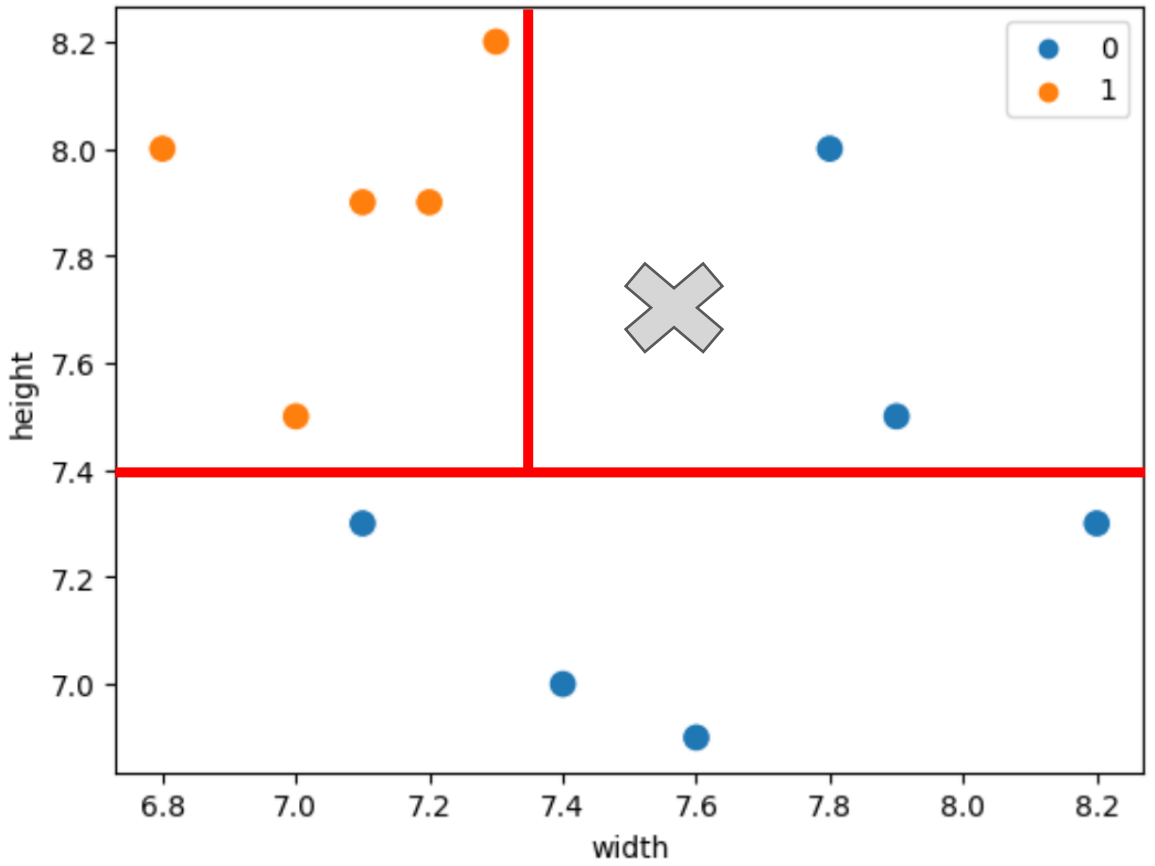
Decision Tree

- Bad Split



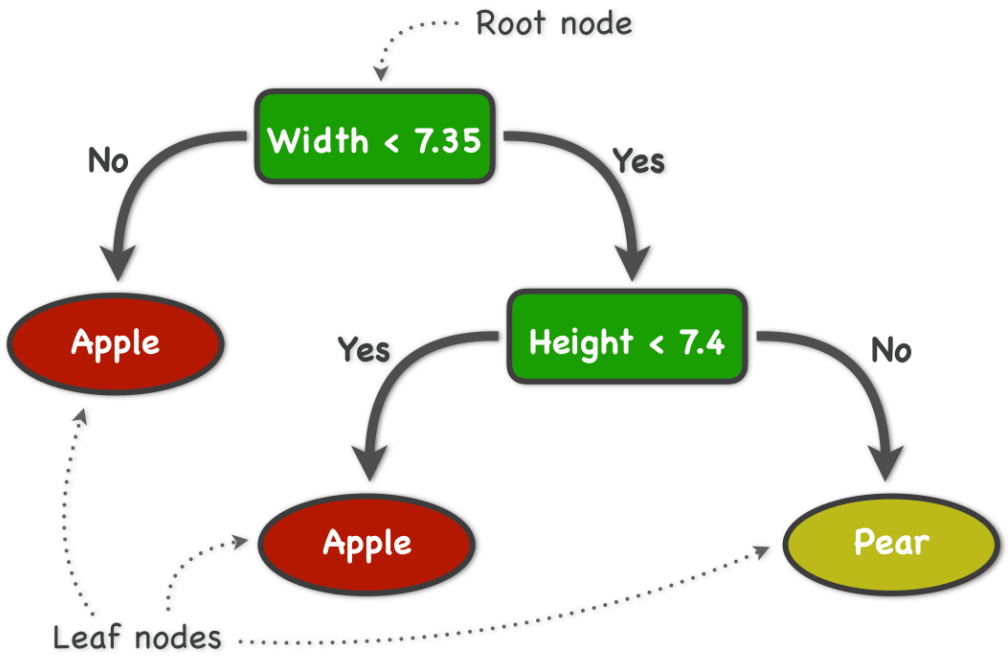
Decision Tree

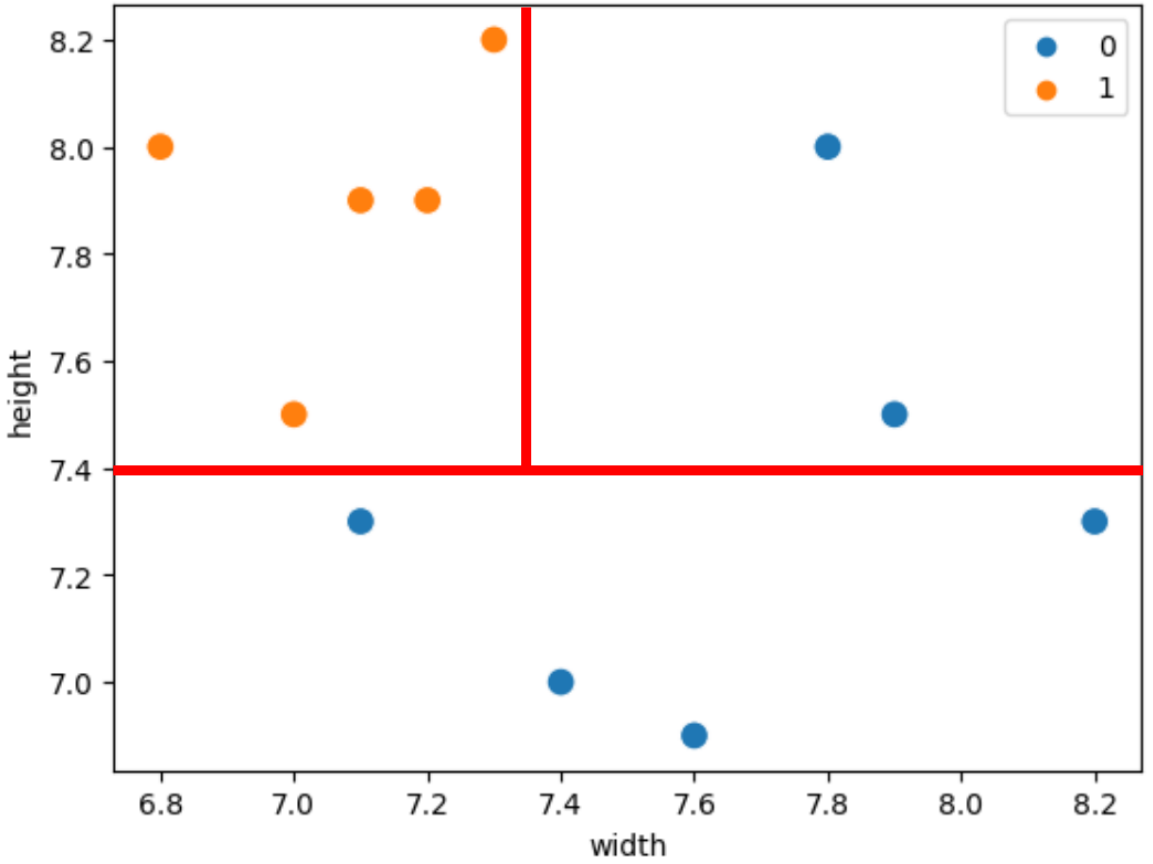
- How can we predict class of new datapoints?



This is a tree!

- Allows reasoning





Decision Tree

- Simple if else structure

```
def get_fruit_class(width, height):
    if width < 7.35:
        if height < 7.4:
            return 'Apple'
        return 'Pear'
    return 'Apple'
```

Width	Height	Fruit
7.1	7.3	Apple
7.9	7.5	Apple
7.4	7.0	Apple
8.2	7.3	Apple
7.6	6.9	Apple
7.8	8.0	Apple
7.0	7.5	Pear
7.1	7.9	Pear
6.8	8.0	Pear
6.6	7.7	Pear
7.3	8.2	Pear
7.2	7.9	Pear

What do we have so far

- **Algorithm** Decision Tree

- **Training Data** applied on Algorithm gives us a Model
  - *That parametrized instance of the Algorithm has wisdom of training data*

- A **Model** we can use for predictions
  - *For a new unseen datapoint infer the class*

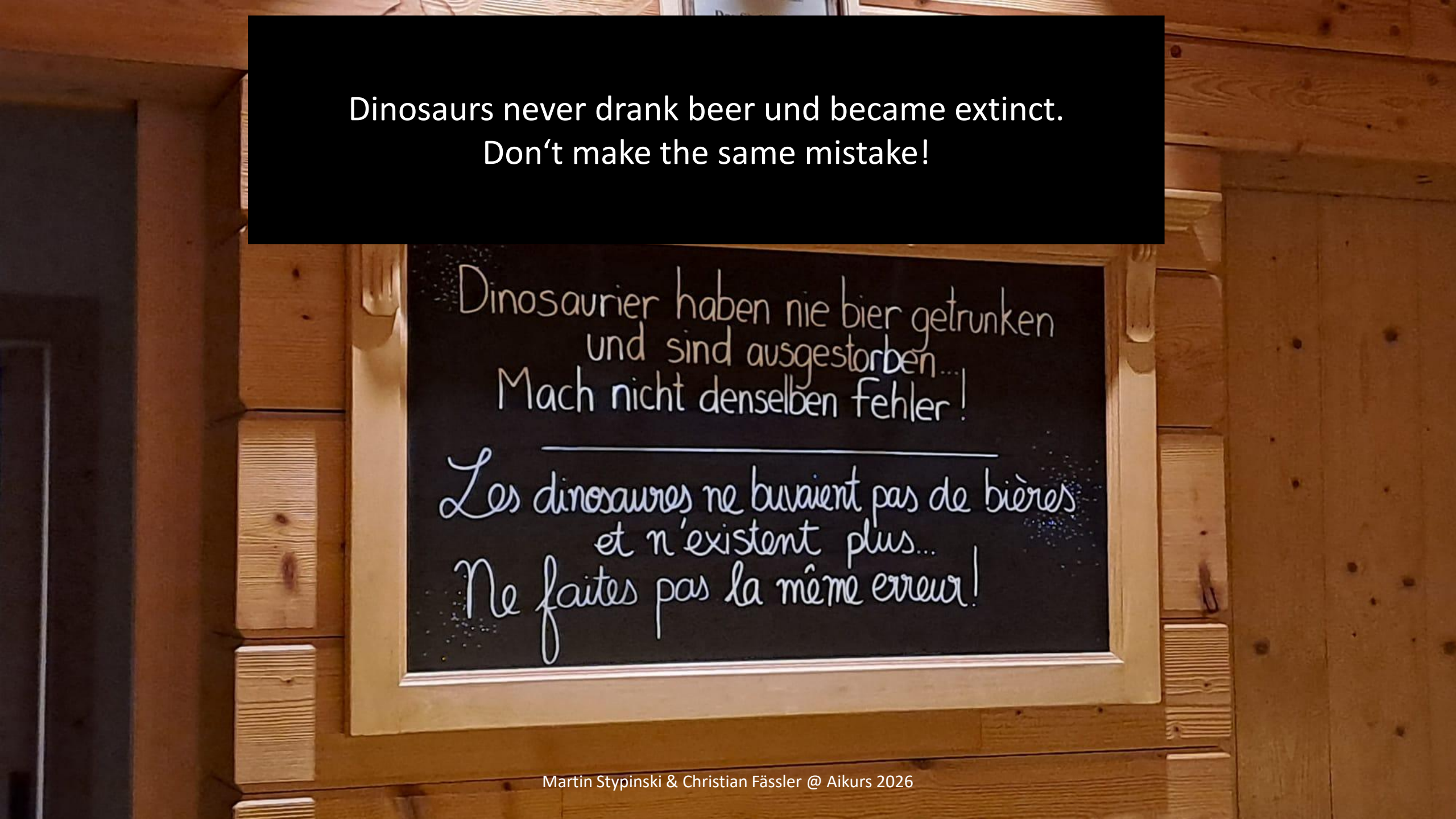
How good is our model?

- **Score** with (unseen) Test Data
  - Test with data we know the correct answer for
  - BUT the model hasn’t been trained with

- **Accuracy** = % of correct predictions

- Not as simple as that
  - False positives vs False negatives → Business impact

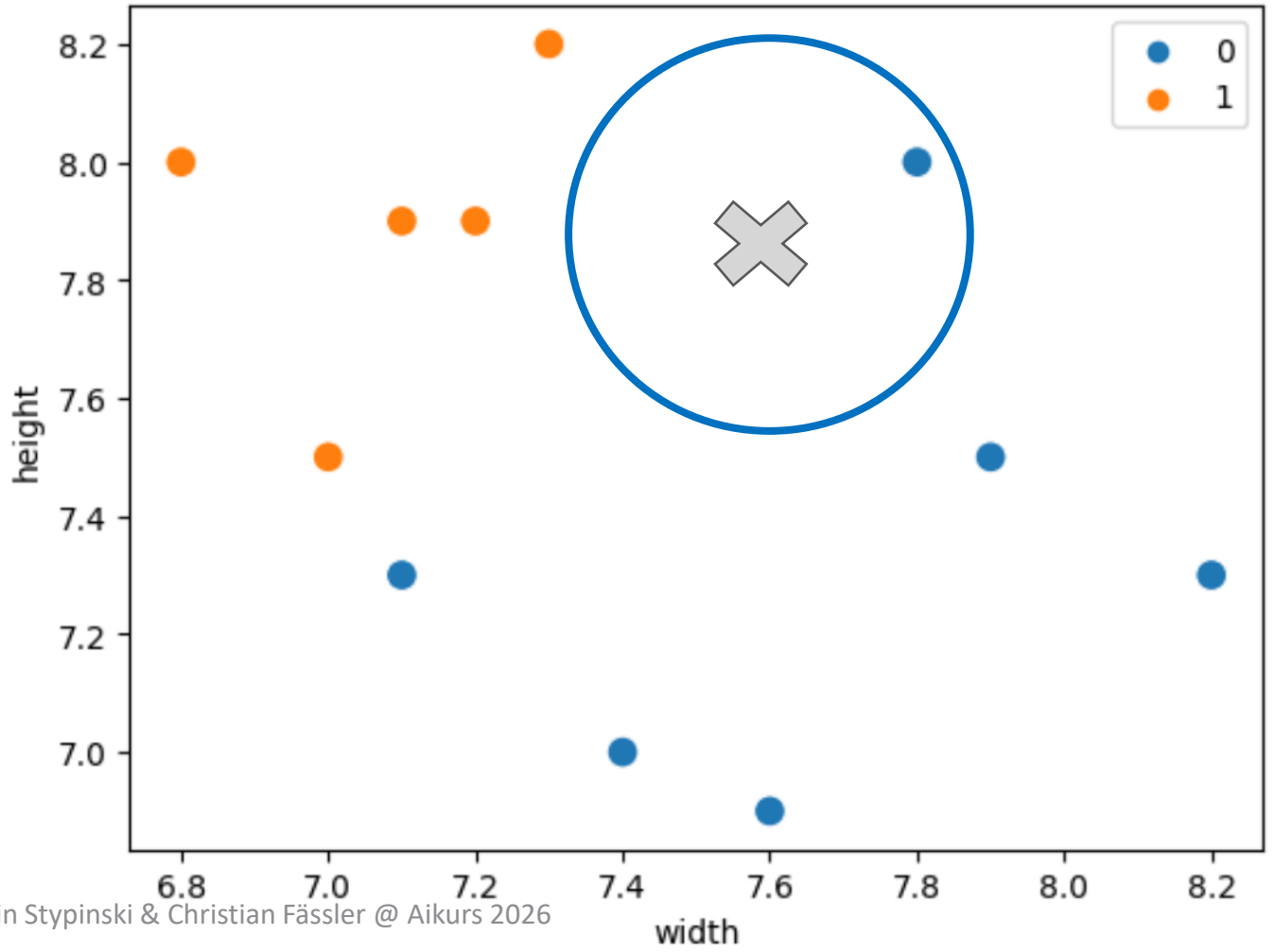
Dinosaurs never drank beer und became extinct.
Don‘t make the same mistake!



Other Algorithm: KNN

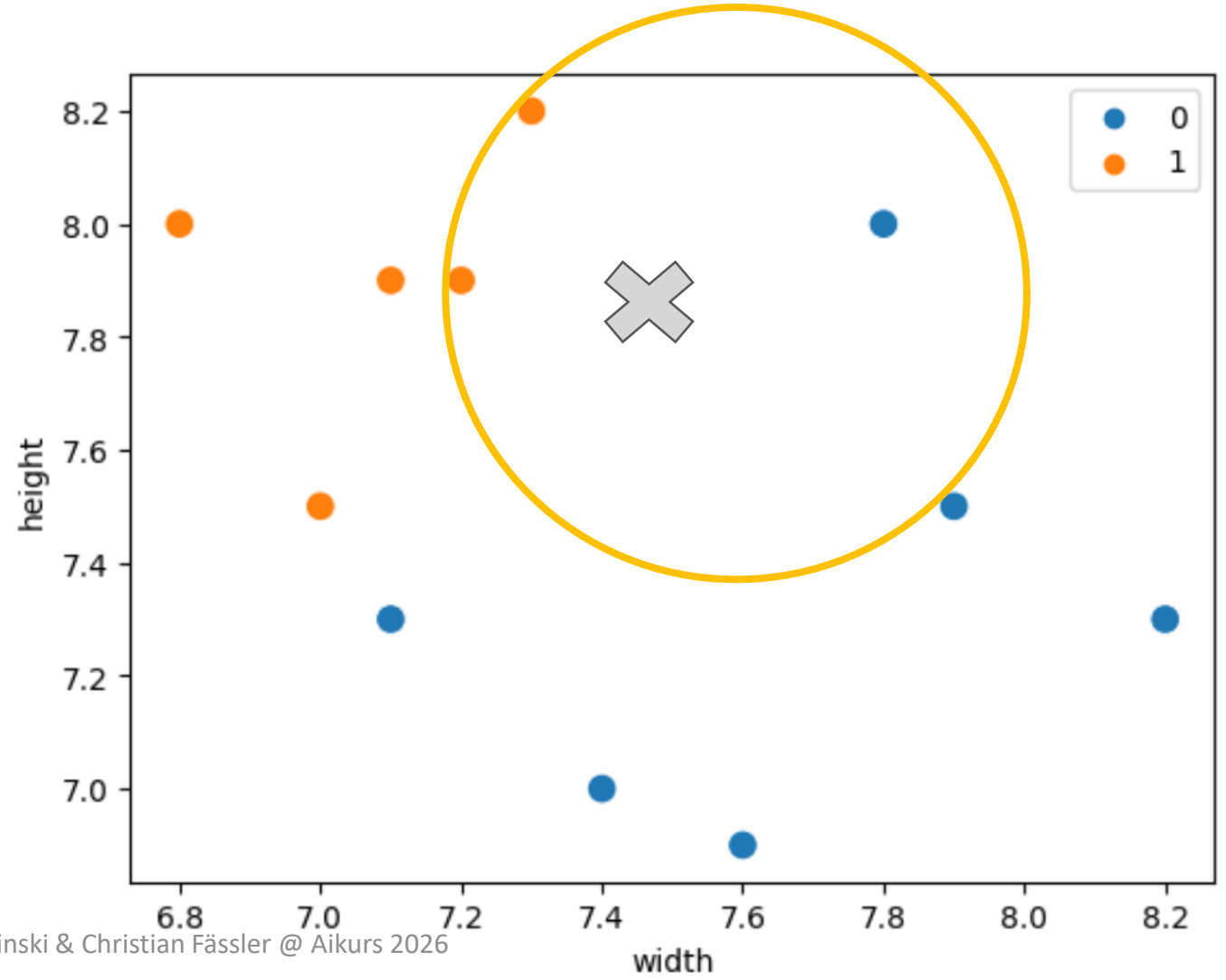
- K Nearest Neighbours
- Look which class the N neighbors have

- K=1 Its an Apple
- K is **Hyperparameter**



KNN

- K Nearest Neighbours
  - K=3 Its a Pear
- K = Hyperparameter
- We need to find best

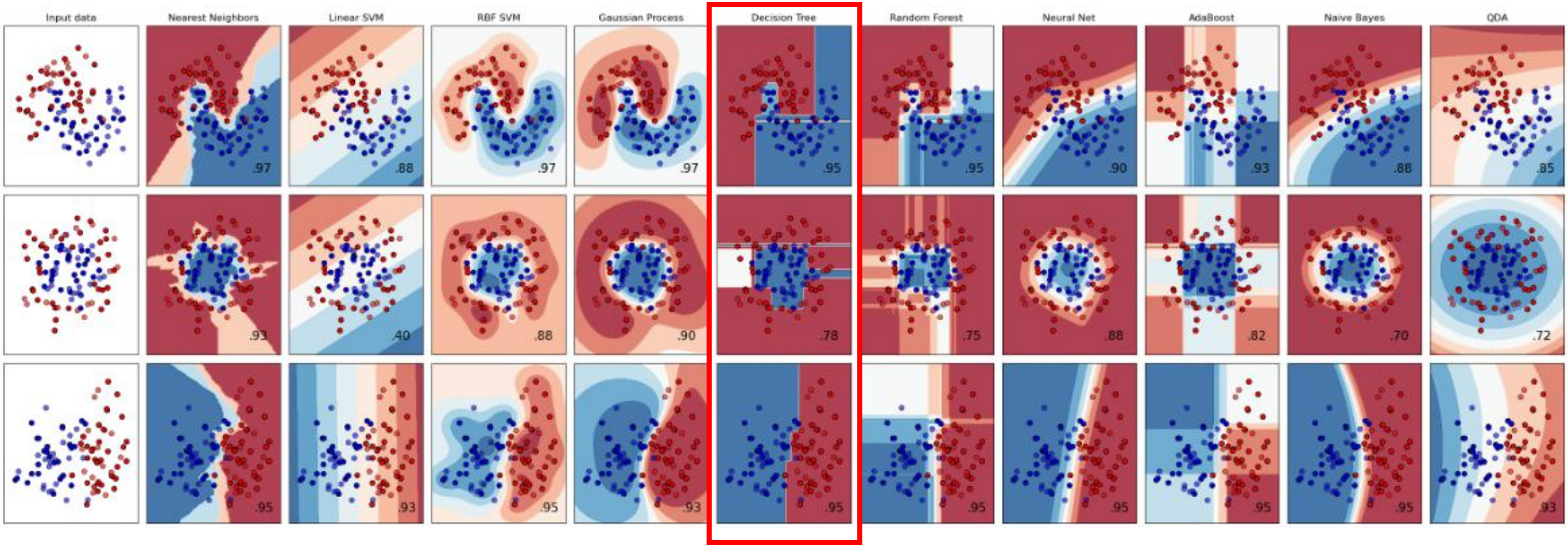


Hyperparameter K

- K = **Hyperparameter**
- We need to find the best - but how?
- Search!
  - Basically we do a search and try out all values and check
  - This is why you need Nvidias and a lot of time ☺

- Example
  - 50’000 Cat images, 50’000 dog images, each 1MB
  - 100GB Data loading and train k times (e.g k=3,5,7,9,11)

Classfication Algorithms



- https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html

Hello World!

```
# Choose the algorithm
classifier = DecisionTreeClassifier()

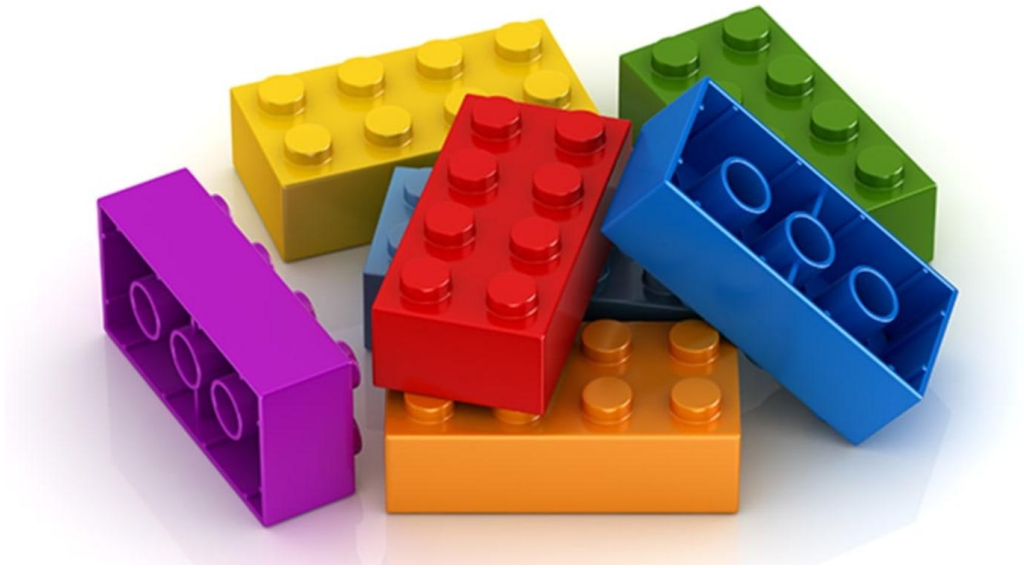
# Do the training
model = classifier.fit(training_input, y=training_correct_output)

# Use trained model to predict on unseen data
prediction = model.predict(new_unseen_data)
```

Demo

- Today it’s like building with LEGO
  - Yeah… almost

- Tooling
  - Python
  - Jupyter Notebooks
  - Scikit-Learn



LAB 1

- Setup
  - Jupyter Notebooks https://jupyter.org/install
  - Use Google Colab https://colab.research.google.com/

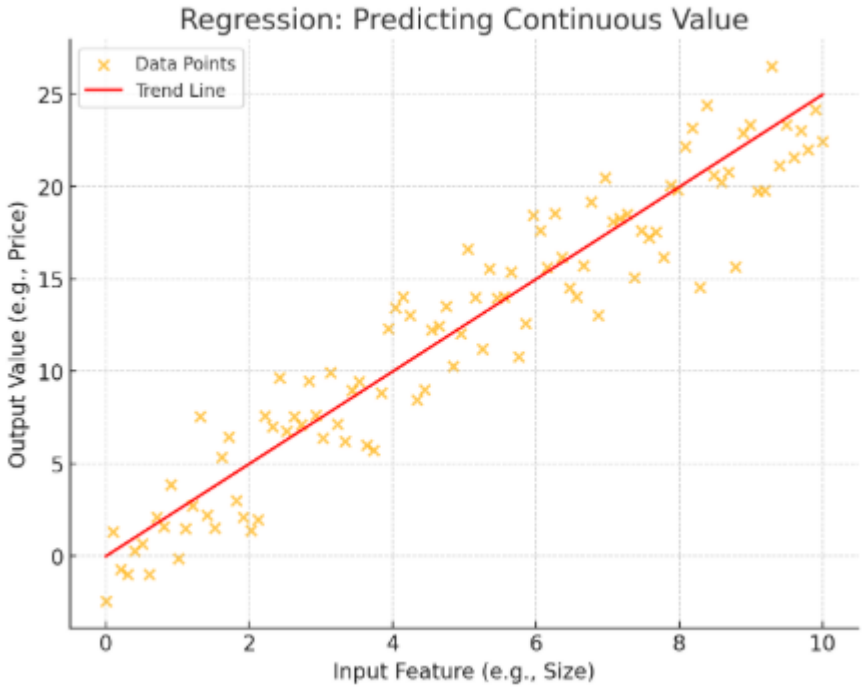
- Download Notebook and let it run
- Some simple tasks are in the Notebook
- https://github.com/cfaessler/mlcon

Problem Types

Type		Output	Examples
Classification	Supervised	Finite set of discrete categories (e.g., "spam" or "not spam"	KNN, DecisionTree
Regression	Supervised	A continuous range of values (e.g., price, weight, speed) [-∞, ∞]	Linear Regression, Polynomial Regression, Gradient Boosting
Clustering	Unsupervised	A set of clusters (e.g., {Cluster 1, Cluster 2, Cluster 3}), with each data point assigned to a cluster.	Kmeans, DBScan, Hierarchical Clustering

Regression

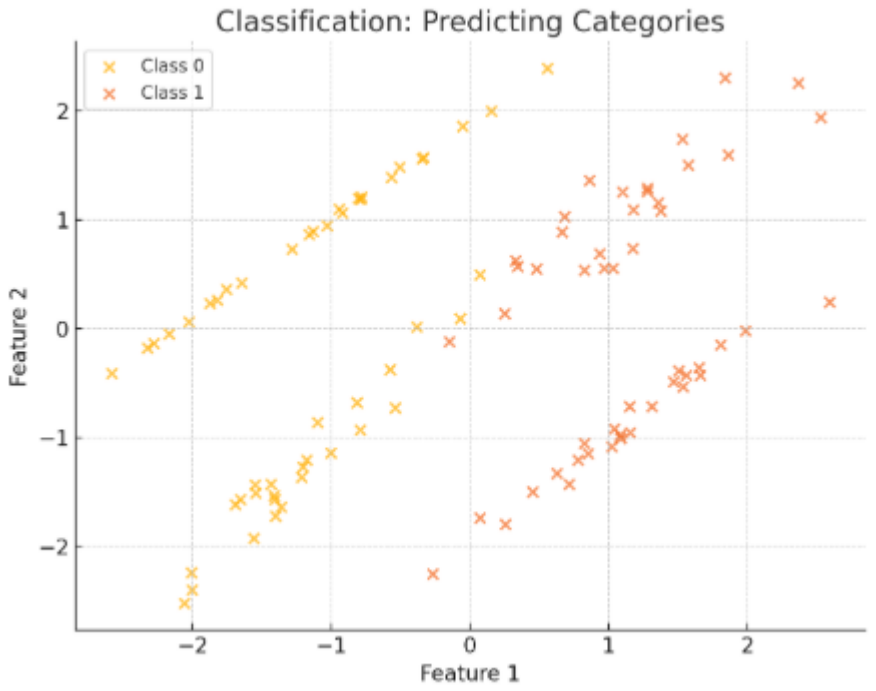
- Predicts a continuous numerical value based on input data
- **Example**: Predicting house prices based on size, location, and other features.



Classification

- Assigns input data to predefined categories or classes

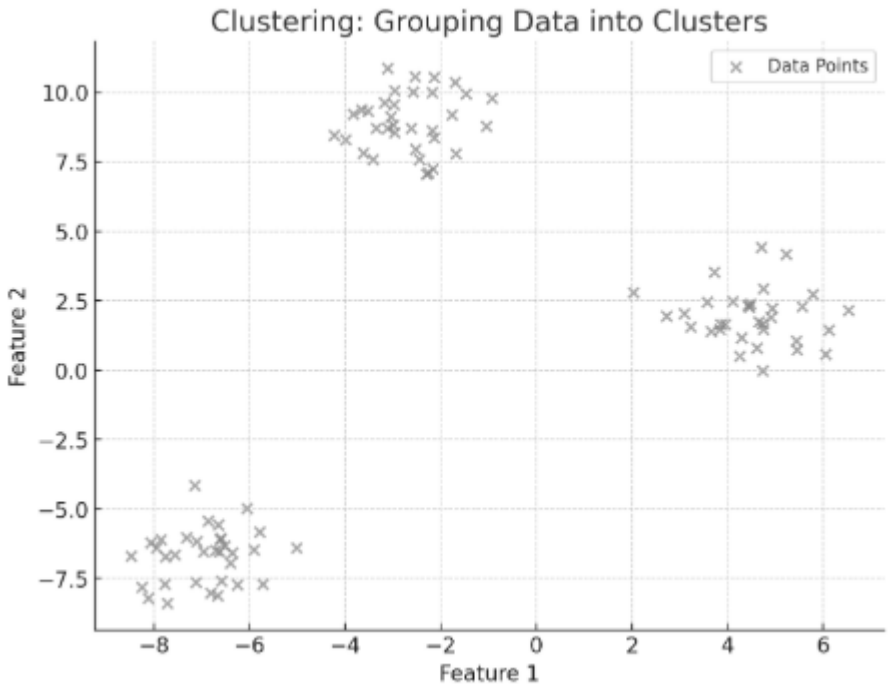
- **Example**: Determining whether an email is spam or not.



Clustering

- Groups similar data points into clusters without predefined labels

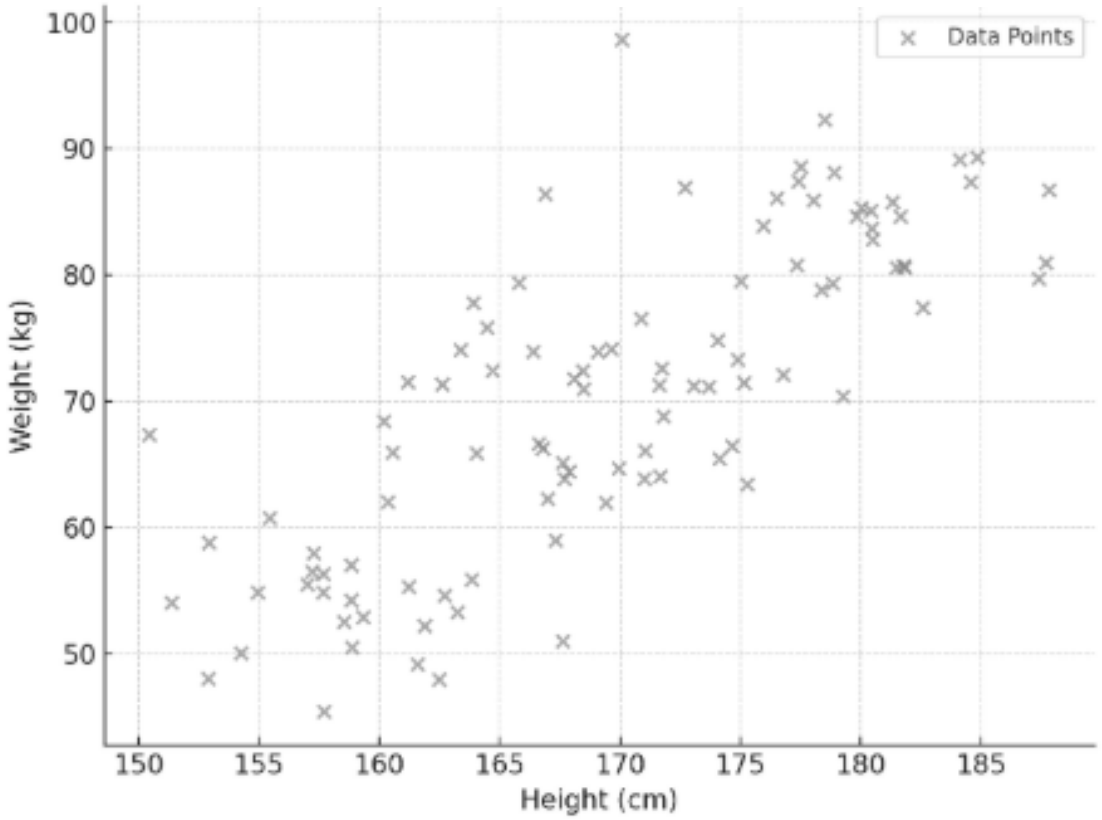
- **Example**: Grouping customers based on shopping behavior.



Example 1

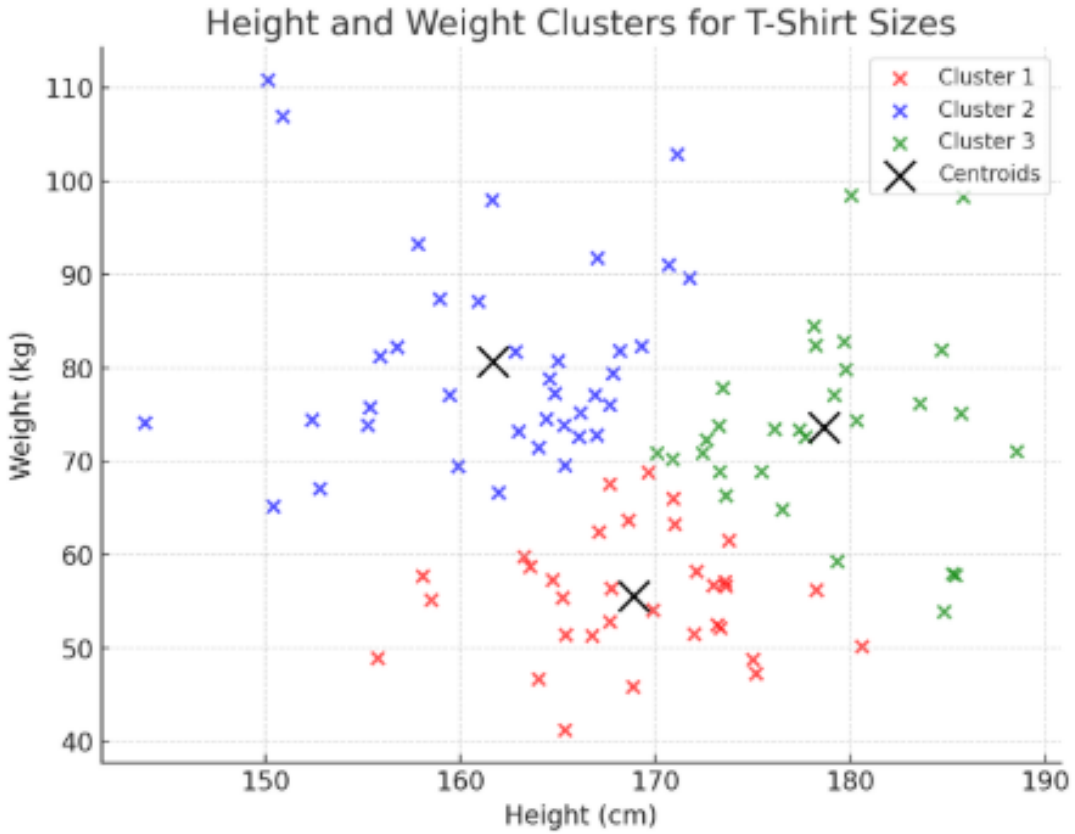
- Given data from 100 employees
- Task: order T-Shirts for all of them in the sizes Small, Medium, Large, Xlarge

- What problem type do i have?
- What is Input / Output?



Clustering

- Group into different clusters (number of available sizes)
- Groups represent people with similar heights and weights
- For each datapoint you know the cluster it belongs too
- For a new datapoint you assign the closest cluster

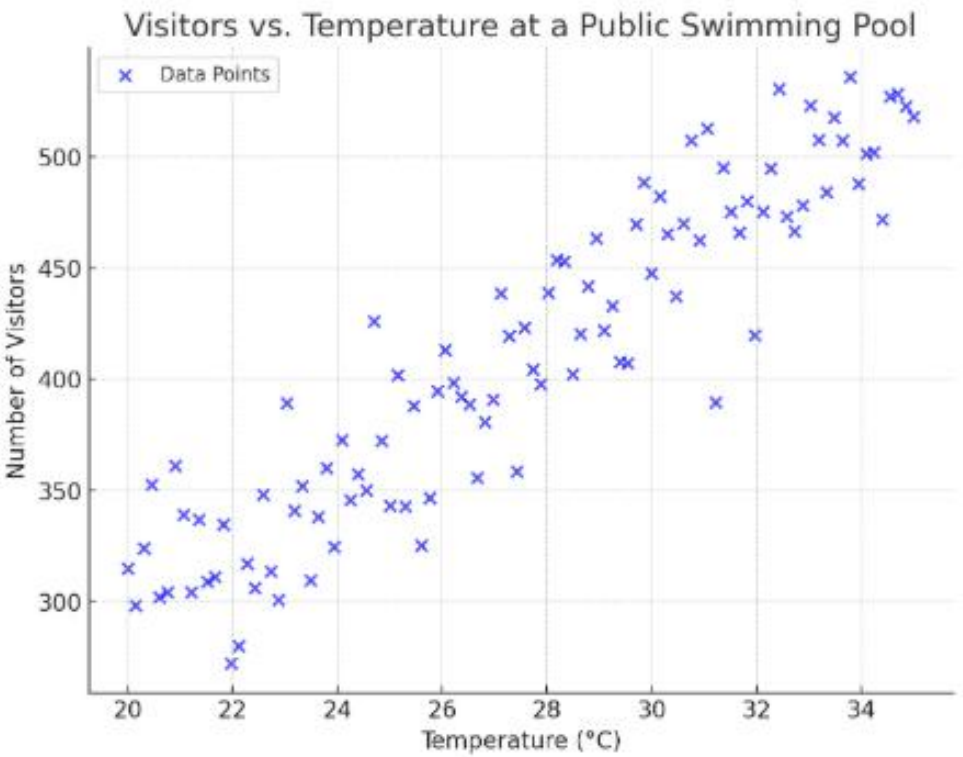


Example 2

- Data from a public swimming pool
  - Number of visitors
  - Outside temperature

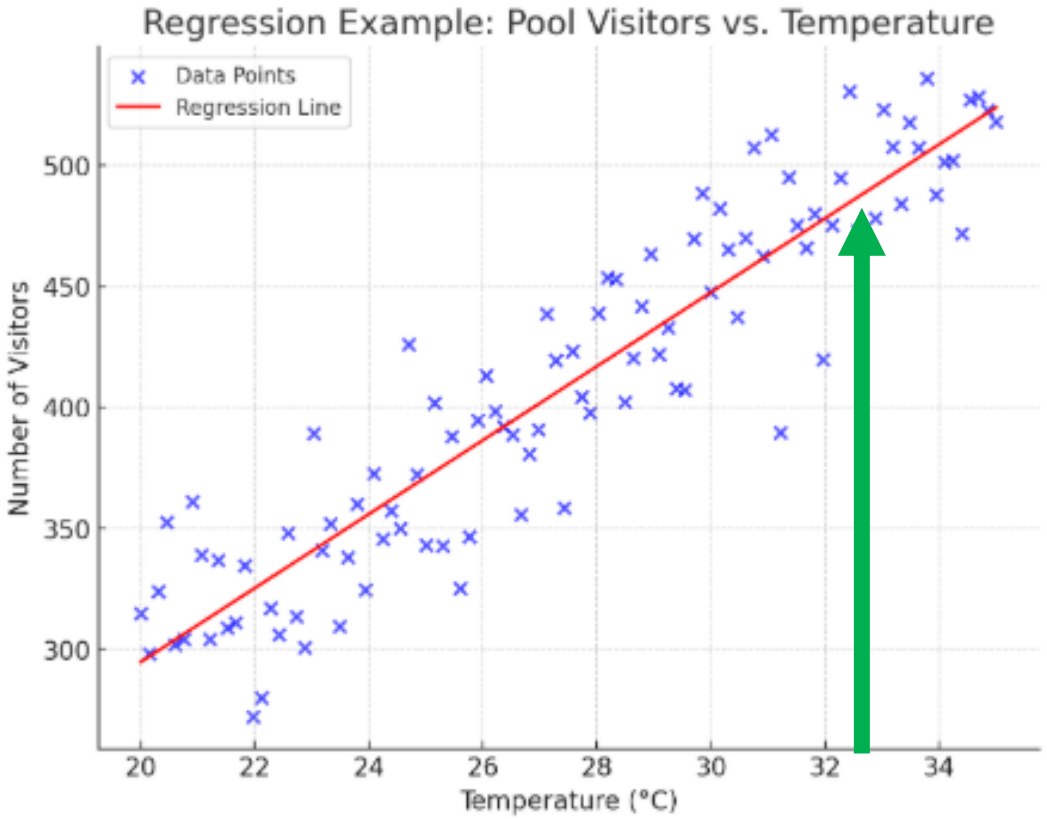
- Task: Predict the number of visitors tomorrow given the temperature forcast

- What problem type do i have?
- What is input/output?



Regression

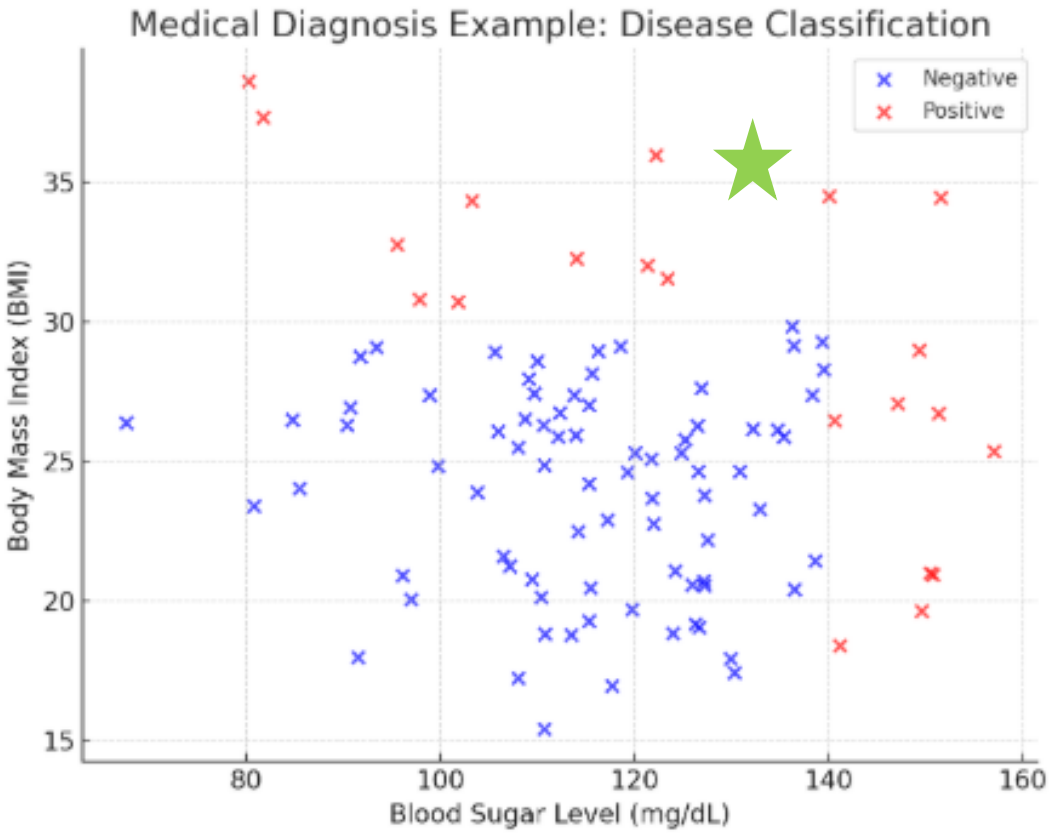
- Train a regression line with the smallest error
- Predict a continuous value of visitors given a temperature



Example 3

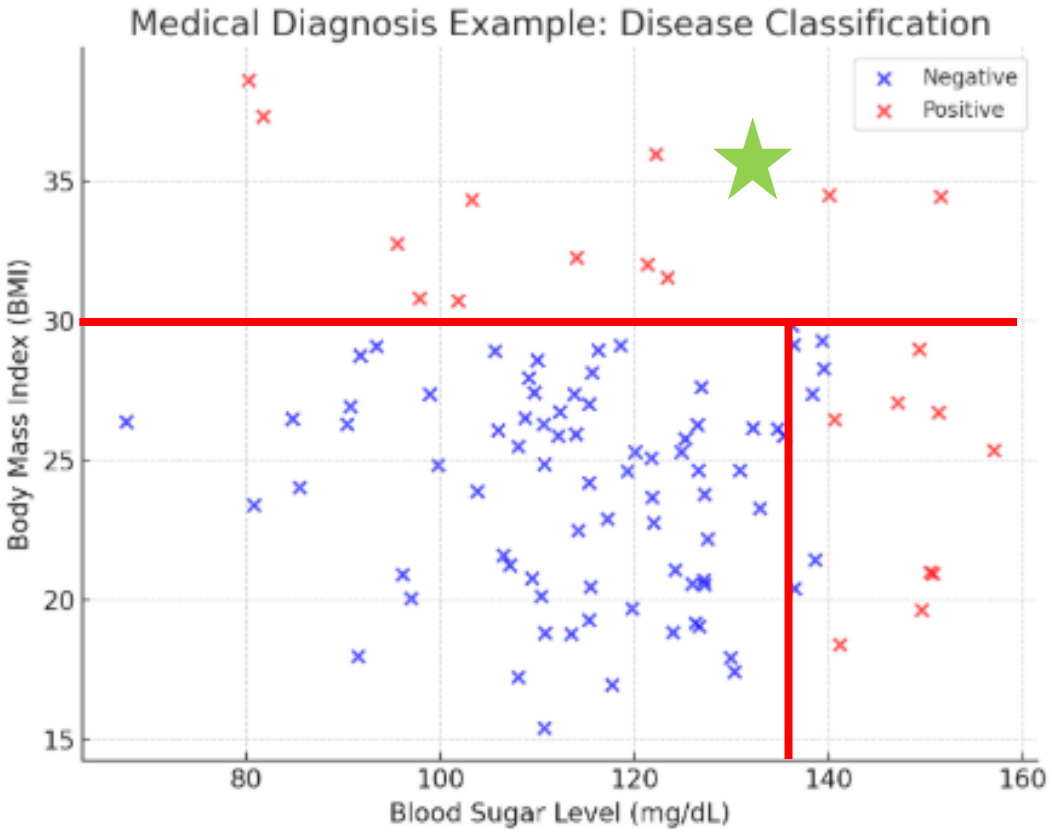
- Medical data
  - Blood sugar level
  - BMI (body mass index)
  - Diabetic yes/no

- Task: I would like to find out if a new person has diabetes or not



Classification

- Learn classes (decision boundaries)
- For a new datapoint predict the class
  - Nearest neighbor
  - DecisionTree



MLOps

- **Deployment**
  - How do we **deploy** and what do we **deploy**?
  - What are the moving parts?
- **Lifecycle**
  - How does it evolve?
- **Integration**
  - How to integrate in our Software Solutions
- **Monitoring**
  - Model Performance

Deployment

- Trained Models can be serialized

```
from joblib import dump, load
# Serialize
dump(model, 'trained-model.joblib')
# Deserialize
loaded_model = load('trained-model.joblib')
output = loaded_model.predict([[50, 50, 100]])
```

Integration: Create an API

- FastAPI with guvicorn

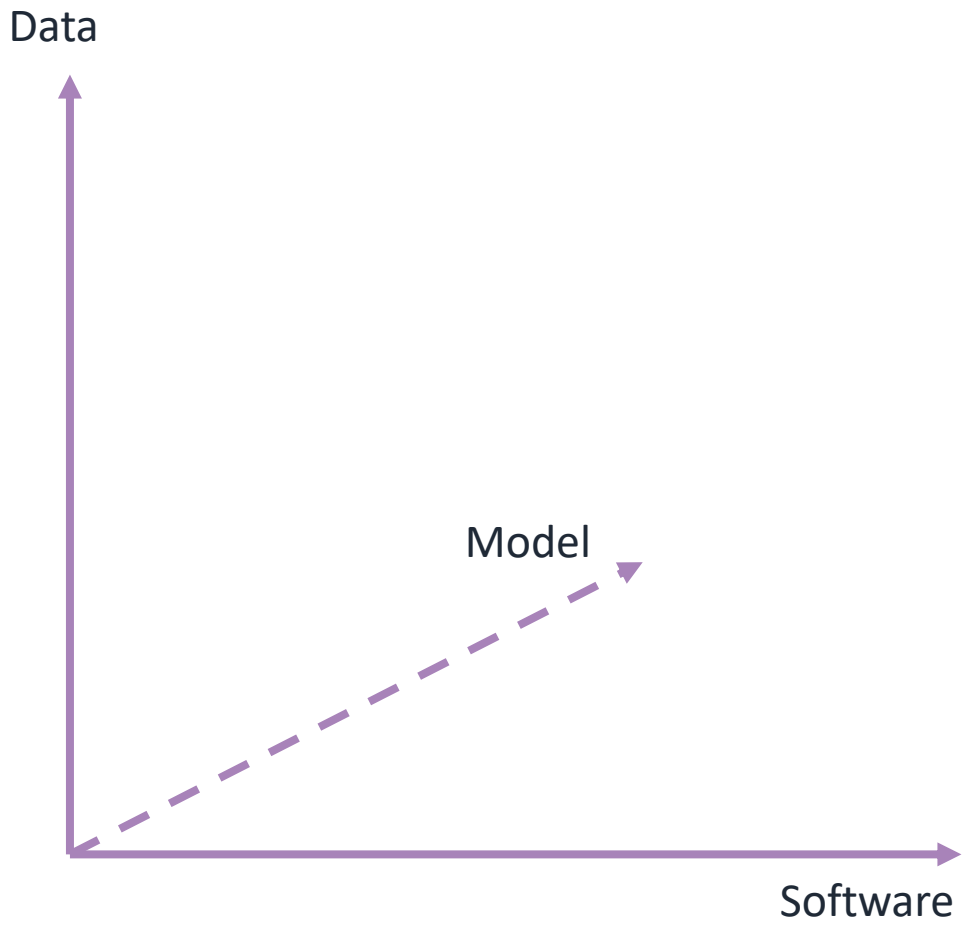
```python
from fastapi import FastAPI
from joblib import load
from sklearn.tree import DecisionTreeClassifier

app = FastAPI()
loaded_model = load('./trained-apple-pears.joblib')


@app.get("/predict/")
def read_item(width: float, height: float):
    output = loaded_model.predict([[width, height]])
    if output[0] == 0:
        return "Apple"
    else:
        return "Pear"
```

What do we need to track

- Code Base of our Software
  - Source Code
  - Dependencies
- Data
  - RAW Data
  - Preprocessed Data
  - https://dvc.org/
- Models
  - Architecture / trained Parameters
  - Hyperparameters
  - Experiments



How to deploy

Training / Experimentation

- Train the model
- Serialize the trained model

Deploy

- Create a Dockerfile
  - Python
  - Serialized model
  - requirements.txt
- Build docker container
- Create REST API using model for inference
- Deploy the API

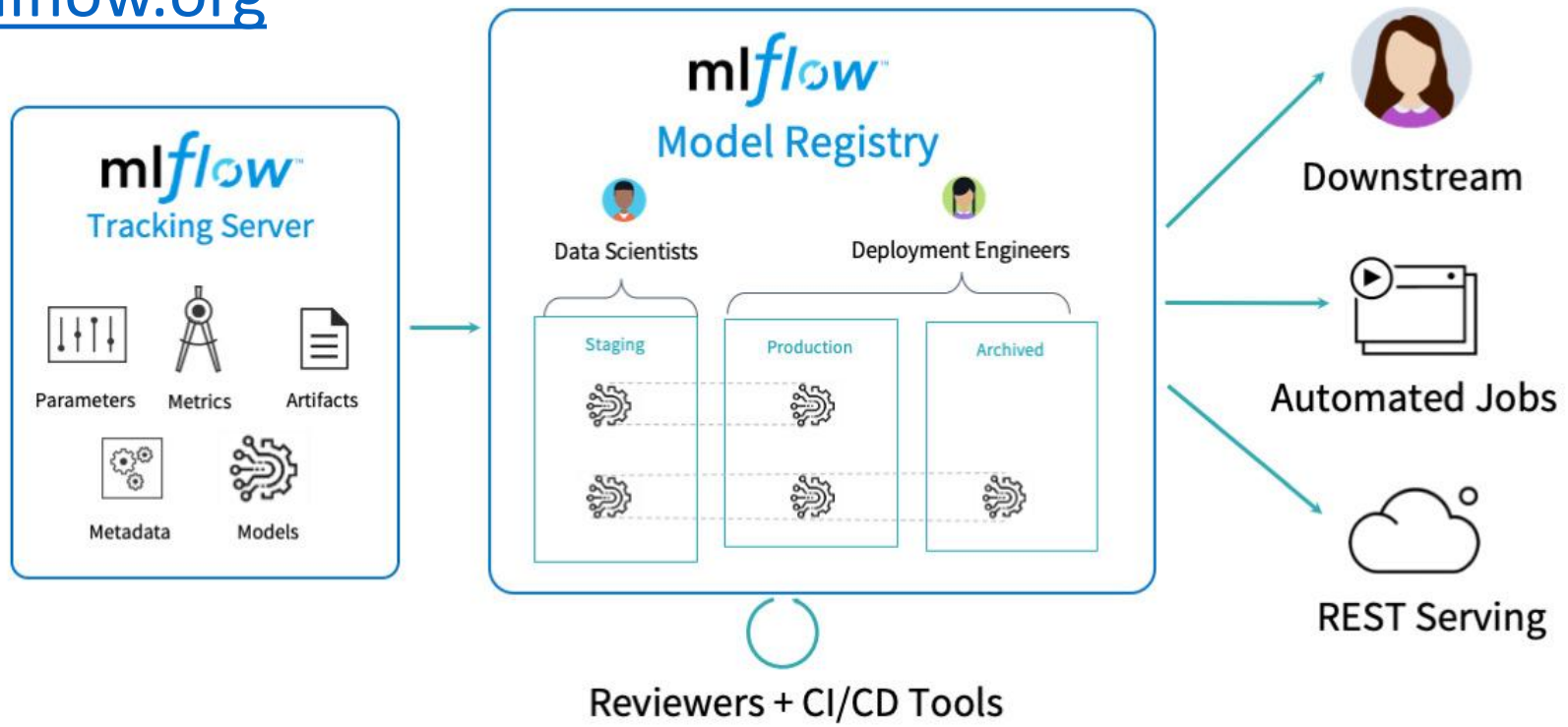
Experiment Tracking

- Keeping track of what we have tried
  - Version the artifacts
- Systematic procedure of discovery and validation of a model
  - Training
  - Evaluation
  - Changing hyperparameters and observe behaviour

- Measure «quality and performance»
- Reproducibility
- torch.manual_seed(3407) is all you need → Paper

Useful Tools

- Weights and Biases https://wandb.ai/site
- MLFlow https://mlflow.org



MLFlow experiment tracking

- What experiments have i done
  - Hyperparameters
  - Performance

```python
import mlflow
import mlflow.sklearn

with mlflow.start_run():
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)

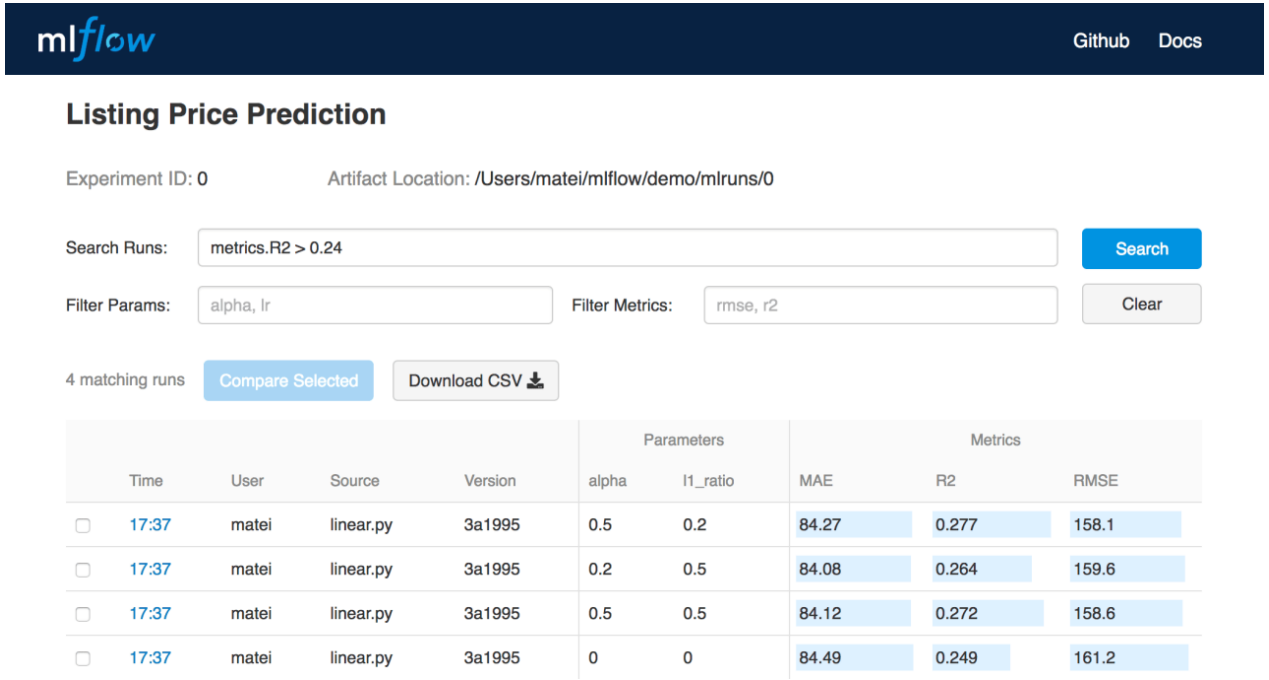
    accuracy = model.score(X_test, y_test)

    mlflow.log_param("n_neighbors", 5)
    mlflow.log_metric("accuracy", accuracy)
    mlflow.sklearn.log_model(model, "knn_model")
```

MLFlow deployment

- As RESTful API

```
mlflow.register_model("runs:/<run-id>/knn_model", "KNNClassifier")
mlflow models serve -m runs:/<run-id>/knn_model -p 5001
```

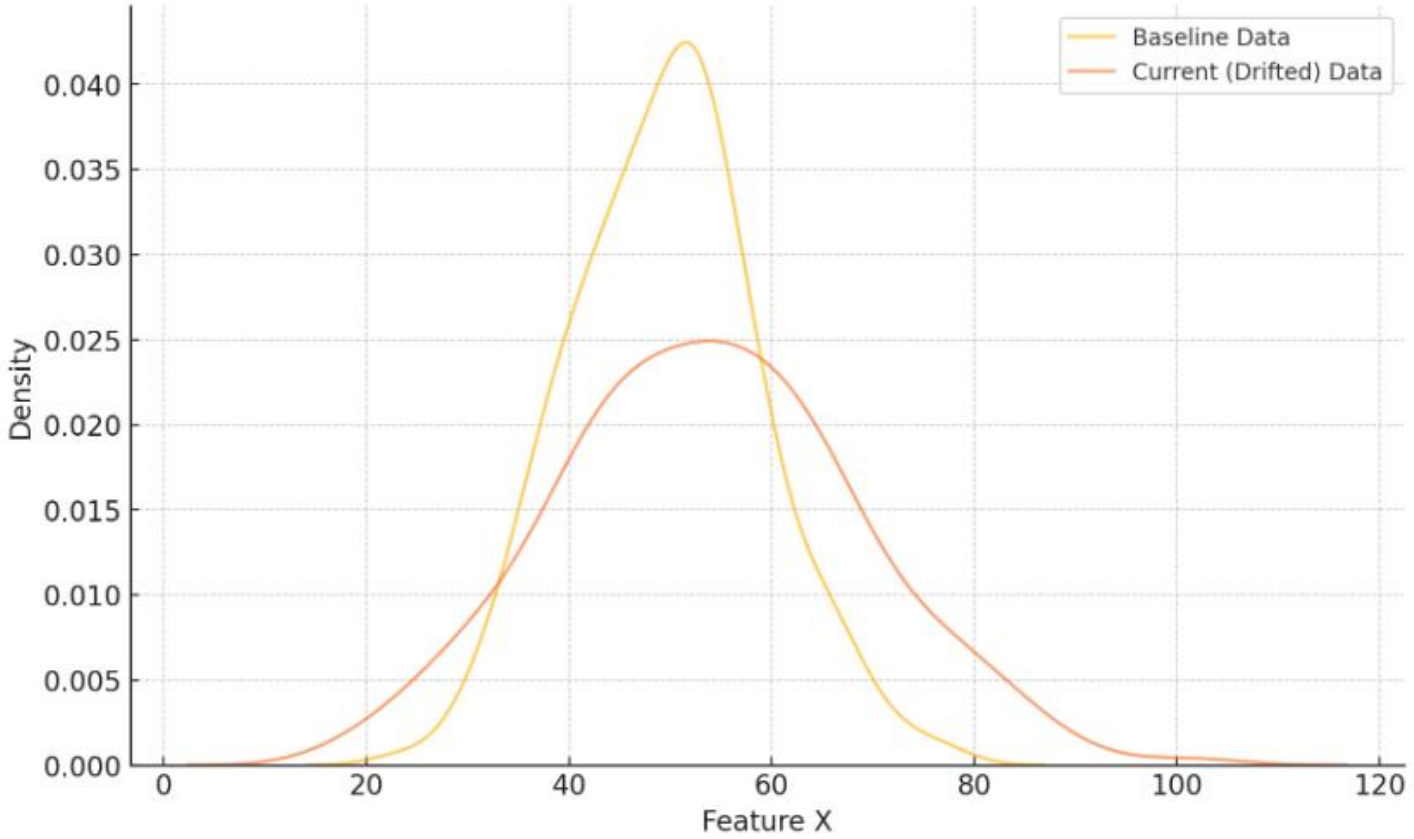


Model Monitoring

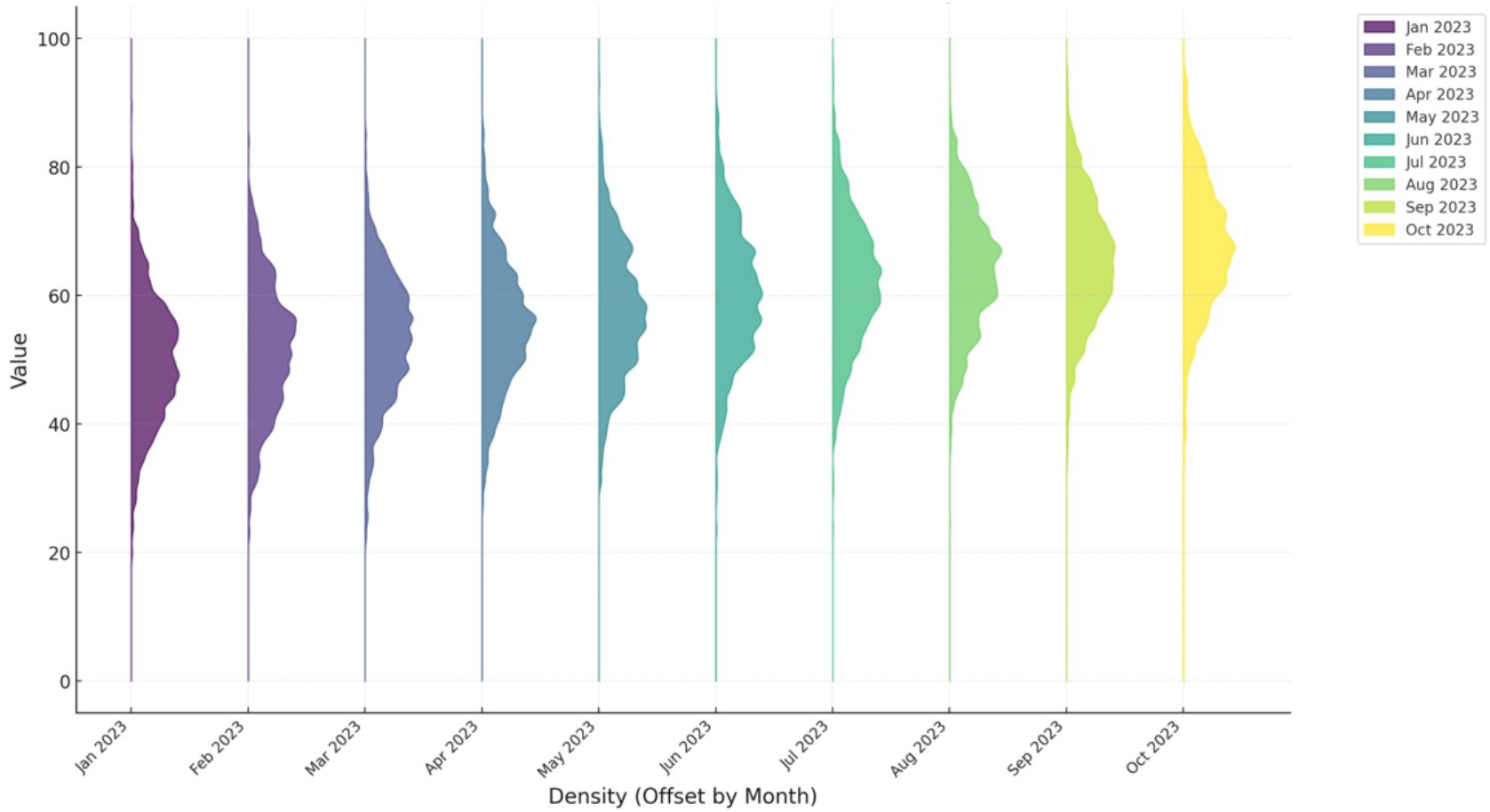
- Monitoring of deployed models
  - What is the performance? (What even means performance?)
  - Problem: I often don’t have labels
  - Delayed Label Handling (Fraud detection: days)
- But
  - I can monitor the data distribution used in production
- Statistical tests

Data Drift

- Statistical properties of the features change over time
- Possible degrade of model performance

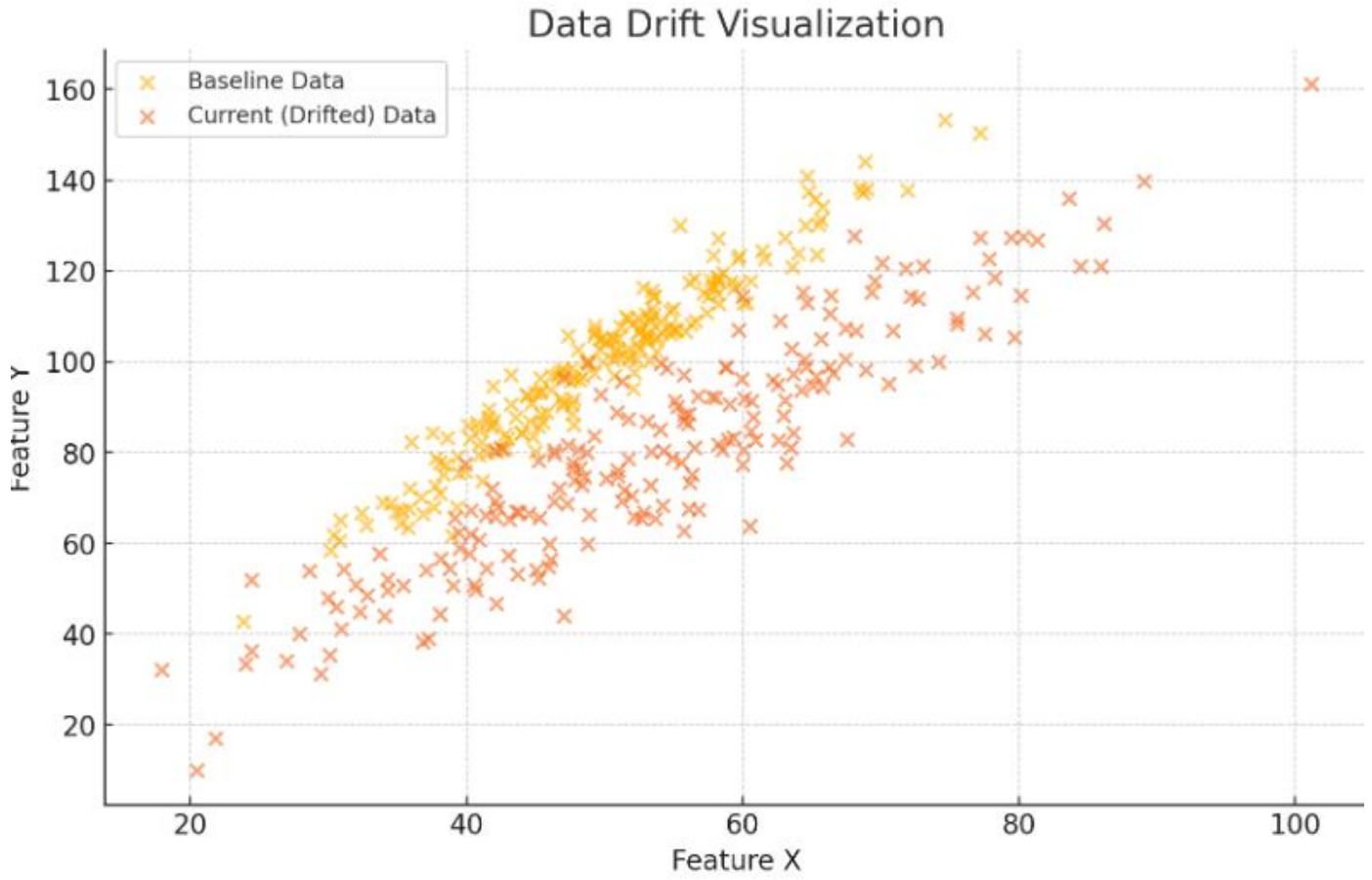


How can I measure that?



Data & Concept Drift

- Covariate Drift
  - Change of the statistical properties of features
- Concept Drift
  - Relationship between features change
- Both possible



Covariate Drift Beispiele

- E-Commerce Plattform
  - **Before Drift** Access mainly via desktop browser
  - **After Drift** Access via mobile apps
  - **Effect** Features like browser type or screen resolution shift, but buying behaviour does not

- Weather prediction
  - **Before Drift** Training with data from last 10 years of 10 cities
  - **After Drift** New city with different climate
  - **Effect** Temperature and Humidity values shift, predictions are not accurate

Concept Drift Examples

- Creditcard fraud detection
  - **Before Drift** Fraudulent transaction are regular and small amounts of money
  - **Nach Drift** Fraudulent transations are onetimers and huge amount of money
  - **Effect** The way how fraud is being done changes

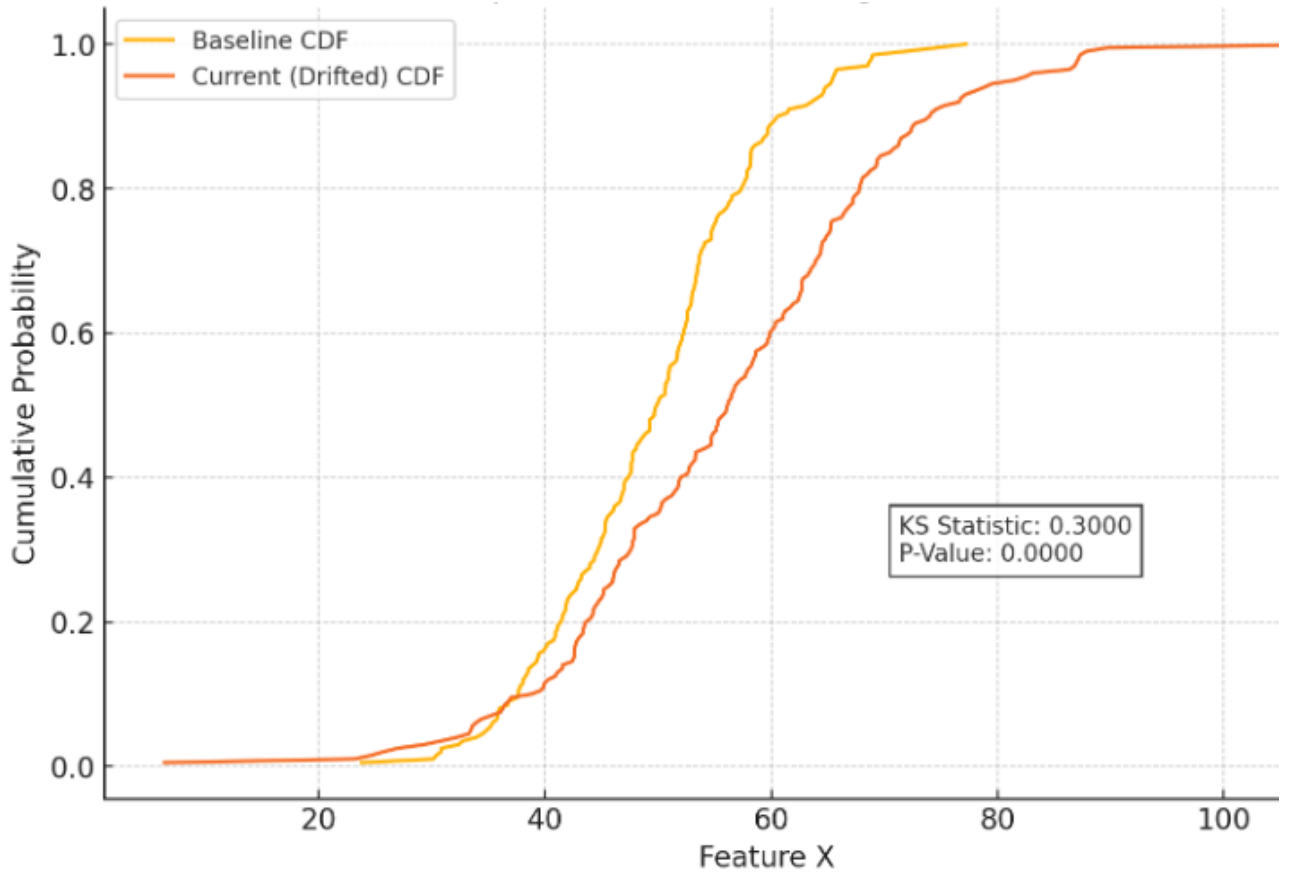
- Movie Recomendation
  - **Before Drift** Recommendations are done based on genres
  - **After Drift** People prefer diversity in recommendations
  - **Effect** users are not happy with recommendations

How can I measure that?

- Compare data distributions
- CDF: Cumulative Distribution Function

- Examples
  - Kolmogorov-Smirnov Test
  - T-Test
  - Chi-Square Test
  - Jensen-Shannon Divergence
  - Wasserstein Distance

- https://www.nannyml.com/

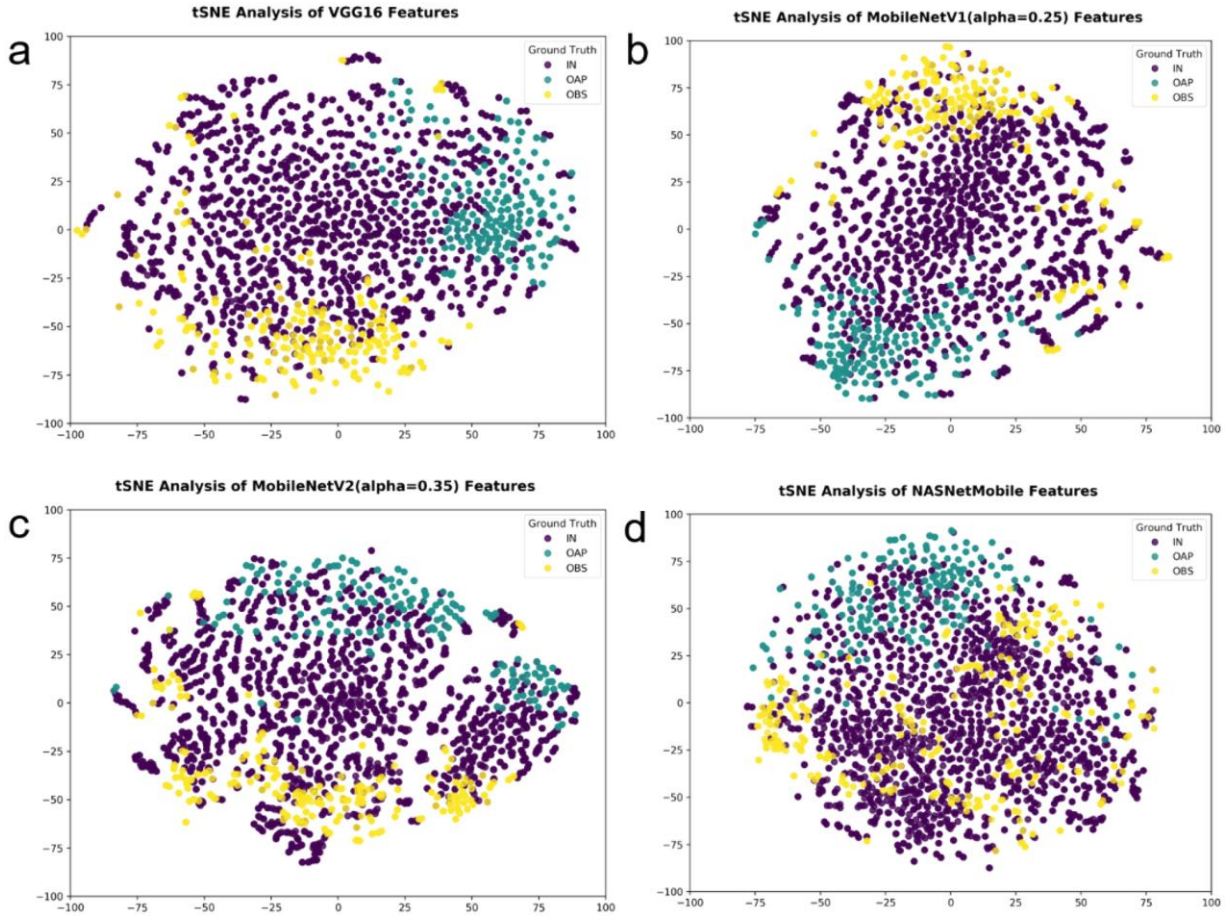


Next steps

- 1. Data handling with Pandas
  - Daten filtering, Aggregate, Load und Store
  - https://pandas.pydata.org/docs/getting_started/index.html
- 2. Data visualization with Seaborn
  - https://seaborn.pydata.org/tutorial.html
- 3. Deal with Classical Algorithms / NN
  - Sklearn https://scikit-learn.org/
  - Pytorch https://pytorch.org/get-started/locally/

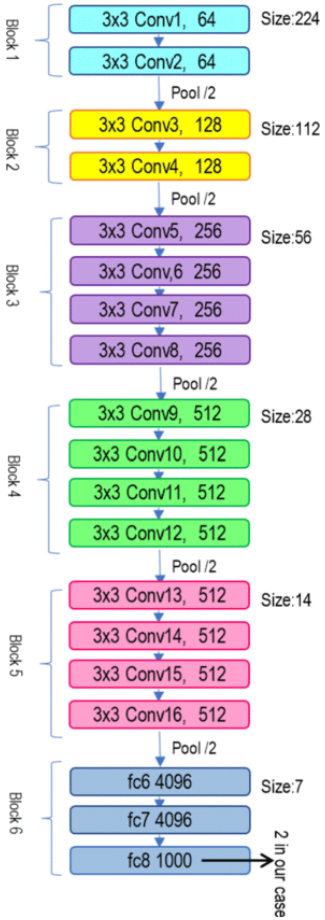
Idea

- Transfer-Learning can be used to fine tune existing models
- Transfer Learning can safe you a lot of money & time
  - Over $100 M (Altman's claim) for GPT4
- «Use a model that can see», fine tune for your use case
- https://poloclub.github.io/cnn-explainer/
- **Popular: LMM fine-tuning!**



1. Pretrained Model

- Object Detection models are often based on Convolutional Neural Networks (CNNs).
- Pretrained models like Faster R-CNN, YOLO and SSD are trained on big datasets like MS COCO or Pascal VOC.
- These datasets contain a lot of images with labeled objects and help models to detect useful features



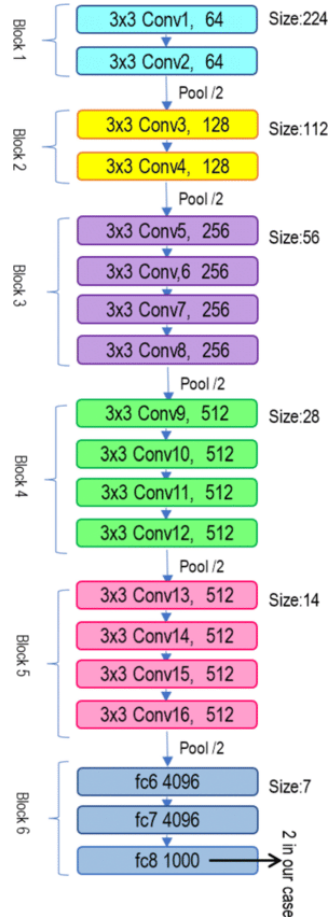


2. Fine-Tuning for specific classes

- An existing model should be fine-tuned for a new application, e.g detect specific objects like tools or animals

- **Freeze early layers:** The early layers stay unchanged, they detect generic features like edges, textures and shapes «seeing»

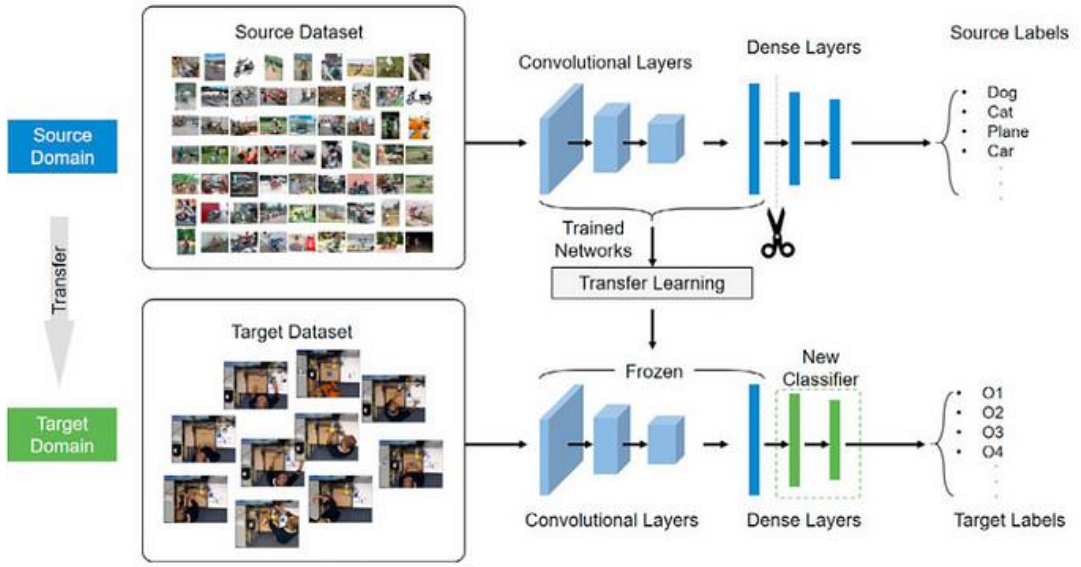
- **Retrain Last Layers:** The last layers are replaced and trained to focus on the specific object types / use case



3. Training on a new dataset

- Our specific dataset will be used for the fine-tuning (training) of our pretrainined model.

- Since the model has already learned to see, it has a lot of generic «knowledge» and only needs a relatively small dataset for retraining



4. Finetuning of Hyperparameters

- Accuracy can further be improved by tuning the hyperparameters.

- Often used
  - Data Augmentation
  - LR Tuning
  - Regularisierung!



5. Test & Evaluate

- Validate optimized model with Testdata

- Thanks to strong pretrained features extractors, the model can «see good» and should learn the new «domain» quickly

- Attention: Verification of the original dataset can be helpful!

Transfer Learning for LLMs

- The LLM does already understand
  - Grammar and Syntax
  - World Knowledge
  - Reasoning patterns
  - Semantic relationships between words and concepts

{"input": "What is the capital of France?", "output": "Paris"}
{"input": “Which is the best ML conference.", "output": “MLCon"}

- Fine-Tuning
  - GPT on customer support chats to build a helpdesk assistant
    - Automate responses
  - GPT on Stack Overflow posts, GitHub repositories, internal codebases
    - Code suggestions

Where to get pretrained models?

- https://huggingface.co/models
- Like github for models



Mini Project

- Use an existing VGG19 CNN to classifiy our own images
  - Apples & Bananas
- This should act as a blueprint to take home

- Full Cirlcle
  - data collection
  - model training (transfer learning)
  - model evaluation
  - deployment with Docker and FastAPI

Transfer Learning VGG16

- ImageNet (ILSVRC) Dataset
  - 1.2 Mio images
  - 1000 classes (hierarchical)
  - Each image has a Label «Dog, Cat, Frog, »



- VGG16
  - 138 Mio parameters
  - Last layer for binary classification 8192 parameters (2 x 4096)
  - Factor of ~16k

1. Data Collection

- Take pictures of apples and bananas using phones
  - At least 20 images of each

- Training (15 pcs/class)
  - /data/train/apples
  - /data/train/bananas

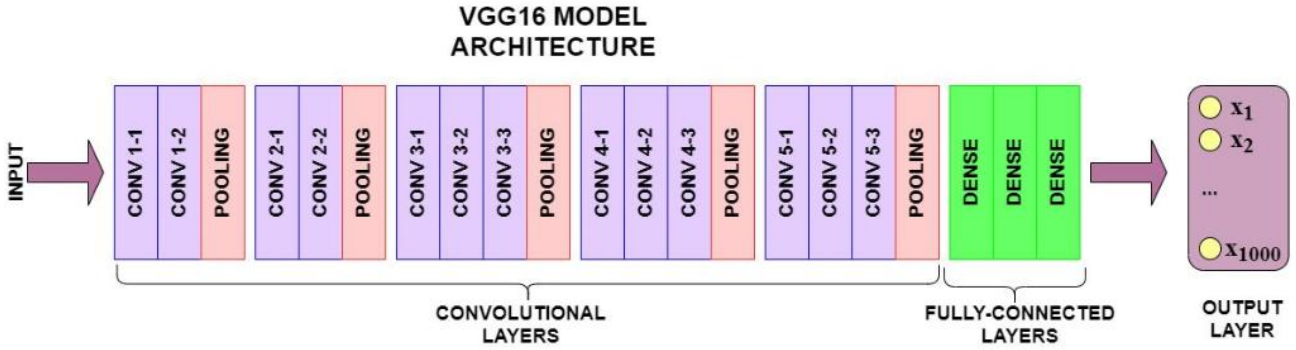
- Test
  (5 pcs/class)
  - /data/validation/apples
  - /data/validation/bananas

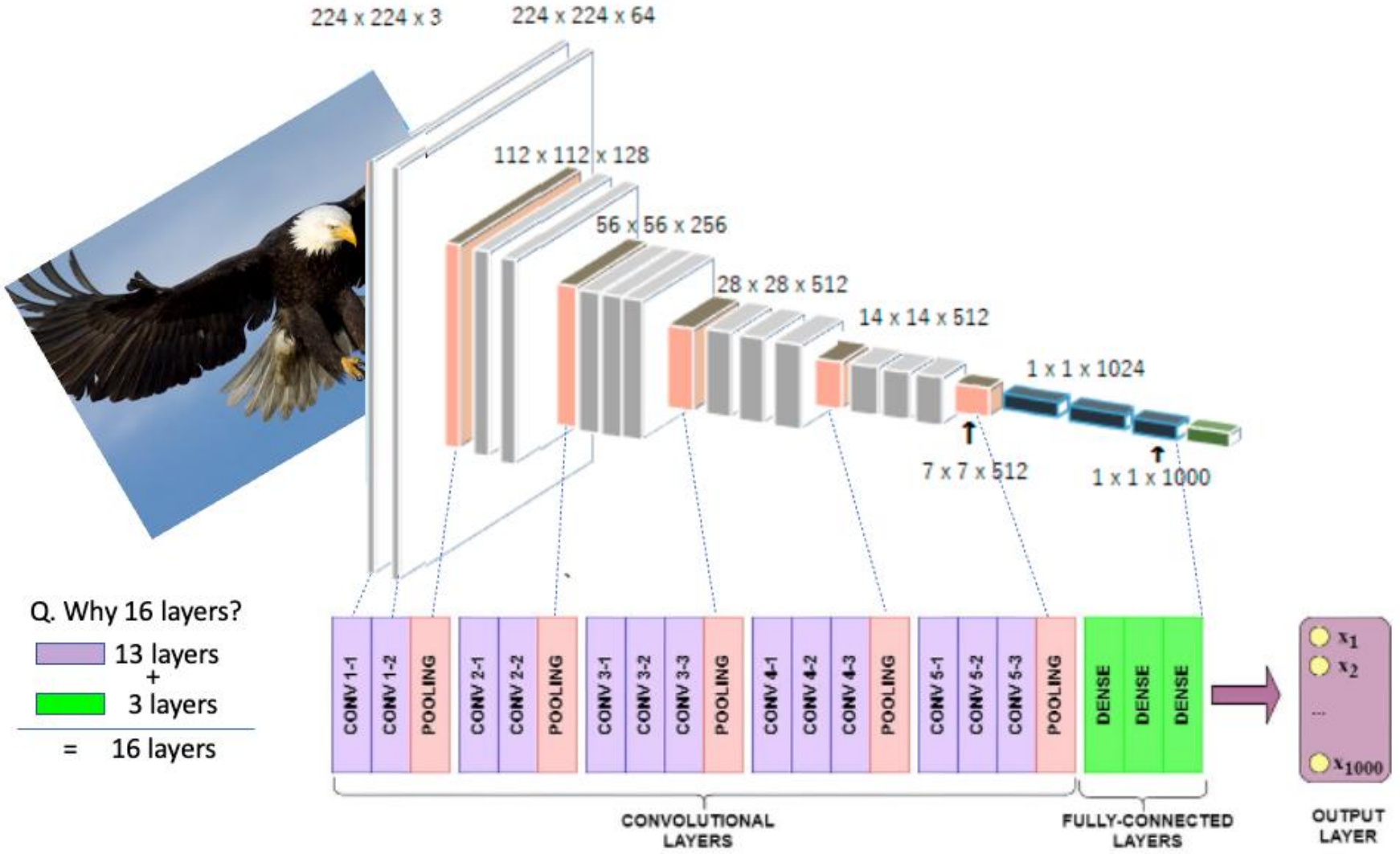
VeeMG



2. Transfer Learning

- Use the provided Notebook
- We fine-tune a VGG16 Neural Network for image classification
  - We freeze all but the the last layers and retrain it
- Steps
  - Preprocessing
    - Standard image size
  - Train
  - Evaluate
  - Save the trained model to a file





- https://www.youtube.com/watch?v=3JQ3hYko51Y
- https://www.youtube.com/watch?v=JboZfxUjLSk

3. Building the API

- Load the trained model when the app starts
- Accept image uploads via a POST endpoint
- Preprocess the uploaded image to match the training data dimensions
- Pass the image through the model and return the predicted class.

Feedback

- Thank you very much!

